



SAPIENZA
UNIVERSITÀ DI ROMA

A Gaussian Splatting Approach to Sewing Pattern Refinement and Garment Alignment

Facoltà dell'informazione, informatica e statistica
Laurea Magistrale in Computer Science

Francesco Palandra

ID number 1849712

Advisor

Emanuele Rodolà

Co-Advisor

Riccardo Marin

Academic Year 2023/2024

Thesis not yet defended

A Gaussian Splatting Approach to Sewing Pattern Refinement and Garment Alignment

Research thesis. Sapienza University of Rome

© 2024 Francesco Palandra. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: palandra.1849712@studenti.uniroma1.it

Abstract

Garment reconstruction refers to the process of generating a mathematical representation of garments from 2D images or 3D point clouds. This task is inherently complex due to the intricate geometries involved, making accurate representations crucial for applications in fashion design, digital avatars, and virtual try-ons. Traditional approaches typically rely on representing garments through sewing patterns—**Francesco: change this explanation: a 2D representation of garment panels akin to those used by tailors**—. Sewing patterns simplify garment structures into flat panels that can be reassembled into the original 3D shape, allowing for both reconstruction and simulation of garments in virtual environments. However, existing methods often depend on real or synthetic datasets, which require expert design knowledge, making data collection both time-consuming and costly. Furthermore, many models fail to provide accurate 3D representations of garments when faced with non-trivial garment shapes and diverse real-world scenarios.

This thesis introduces TailorGS, a novel approach designed to refine sewing patterns and address the limitations of current garment reconstruction methods. While existing models often struggle with out of distribution garments—those not seen during training or that deviate from common garment structures—TailorGS leverages Gaussian Splatting to enhance the accuracy of garment reconstruction. Gaussian Splatting is used to represent and refine the 3D garment mesh, enabling the model to iteratively adjust and align the garment’s structure with the expected output. By applying this technique, TailorGS significantly improves the fidelity of sewing pattern extraction from 3D point cloud data, overcoming the inability of other models to generate trustworthy representations of complex garment shapes. Through extensive experimentation, TailorGS demonstrates superior performance in producing accurate garment patterns from complex, real-world garment geometries, marking a significant step forward in the field of garment reconstruction.

The key contribution of TailorGS lies in its ability to refine and align the sewing pattern outputs of existing models by integrating physical simulation and Gaussian Splatting. First, a physical simulator is used to generate a 3D model from the sewing patterns. Subsequently, Gaussian Splatting is applied to adjust a subset of points sampled from the garment’s mesh, shifting them towards their ground truth positions. This process ensures that the final sewing patterns are highly precise and aligned with the garment’s true structure. TailorGS thus presents a robust solution to the challenges of dataset limitations and complex garment geometries, offering a more versatile and accurate tool for garment reconstruction and sewing pattern extraction.

Contents

1	Introduction	1
2	Related Work	4
2.1	Sewing Patterns	4
2.2	Point Cloud	4
2.3	Generative approaches	4
2.4	Other approaches (TBD)	4
2.5	Garments datasets	4
3	Background	5
3.1	Sewing Patterns	5
3.1.1	Representation of Sewing Patterns	5
3.1.2	Mathematical Representation	6
3.1.3	Advantages of Sewing Patterns in Garment Reconstruction .	7
3.1.4	Panel Representation in JSON Format	8
3.2	SMPL	8
3.3	Warp	9
3.3.1	UV Mapping	10
3.4	Technical Works	10
3.4.1	Neural Tailor	11
3.4.2	SewFormer	11
3.5	Gaussian Splatting	11
3.5.1	Overview of Gaussian Splatting	11
3.5.2	How Gaussian Splatting Works	11
3.5.3	Mathematical Representation	12
3.5.4	Advantages of Gaussian Splatting	12
3.5.5	Applications in Garments	13
3.5.6	Challenges and Limitations	13
4	Methodology	14
4.0.1	Nomenclature	14
4.0.2	Problem formulation	16
4.0.3	Pipeline Overview	16
4.1	Sewing Pattern Reconstruction	18
4.1.1	Setting up Gaussian Splatting	18
4.1.2	Sewing Pattern Mesh Extraction	22

4.1.3	The Gaussian Splatting Reconstruction	26
4.1.4	Towards the Garment Mesh	30
4.2	Refinement in GS	34
4.2.1	Opacity Problem	35
4.3	Optimizations	38
4.4	Correspondance Mapping	41
4.4.1	Flatting Techniques	42
4.5	No backbone	45
4.6	Implementation Details	45
5	Results	46
5.1	Validation Analysis	46
5.1.1	Plots	46
5.1.2	Disappeared	46
5.1.3	Gaussians Displacement	46
5.2	Applications	46
6	Applications	47
6.1	Industrial Use Cases	47
6.2	3D garment Simulations	47
7	Conclusion	48
7.1	Key Contributions	48
7.2	GS in the garment reconstruction field	48
7.3	Limitations of TailorGS	48
7.4	Future Works	48
7.4.1	Flattening approach	48
	Bibliography	49
	Figure	51
	Ringraziamenti	51

Chapter 1

Introduction

Francesco: still a draft Garment reconstruction is an emerging field within deep learning that focuses on generating accurate mathematical representations of clothing items from visual data such as 2D images or 3D point clouds. Despite its nascent status, garment reconstruction holds significant potential due to its wide range of applications in industries like fashion design, virtual reality, digital avatar creation, and virtual try-on systems. Accurate garment reconstruction enables designers to visualize and manipulate clothing digitally, enhances the realism of digital avatars in gaming and virtual environments, and provides consumers with interactive virtual fitting experiences, thereby bridging the gap between physical and digital fashion.

At its core, garment reconstruction entails the creation of detailed and precise 3D models of clothing items from visual inputs. This involves capturing the complex geometries, folds, and textures of garments, which are challenging due to the inherent variability in fabric types, styles, and fitting. The reconstruction process typically leverages advanced machine learning techniques to interpret and translate 2D images or 3D point clouds into coherent 3D models that faithfully represent the original garments. One of the fundamental components in garment reconstruction is the use of sewing patterns—2D representations of garment panels akin to those used by tailors. Sewing patterns simplify garment structures into flat panels that can be reassembled into the original 3D shape, facilitating both the reconstruction and simulation of garments in virtual environments. This representation is particularly convenient as it bridges the gap between 2D visual data and 3D garment models, making the reconstruction process more interpretable and manageable.

Creating high-quality synthetic datasets is crucial for training robust garment reconstruction models. A common approach involves using the Skinned Multi-Person Linear (SMPL) model, a parametric model that generates realistic human body shapes and poses. By integrating SMPL with a draping simulation process, researchers can create diverse and detailed 3D garment models that accurately reflect how clothing interacts with various body types and movements. This synthetic data generation process allows for the creation of large, annotated datasets without the need for extensive manual data collection, thereby addressing the challenges of dataset scarcity and variability in real-world scenarios.

While existing garment reconstruction methods have made significant strides, they often encounter limitations in handling complex or out-of-distribution garment shapes due to the constraints of their training datasets and inherent mathematical challenges. Many models fail to provide accurate 3D representations of garments when faced with non-trivial garment shapes and diverse real-world scenarios, particularly when dealing with garments that exhibit unique features not present in the training data. To address these shortcomings, this thesis introduces TailorGS, a novel approach that refines sewing patterns to achieve high-fidelity garment reconstruction. TailorGS employs Gaussian Splatting, a technique used to represent and manipulate 3D point clouds, to enhance the accuracy of sewing pattern extraction and alignment. The approach consists of three main modules: first, a draping process utilizes a physical simulator to generate a 3D garment model from initial sewing patterns provided by existing models (e.g., SewFormer); second, the Gaussian Splatting module adjusts a subset of points sampled from the garment mesh, refining their positions to better align with the ground truth garment geometry; and third, UV Projection maps the refined point movements back onto the 2D sewing patterns using the garment mesh’s UV map, ensuring precise alignment and accuracy in the final sewing patterns. This multi-step refinement process enables TailorGS to overcome the limitations of existing models, particularly in scenarios involving complex garment geometries and out-of-distribution examples. By integrating physical simulation with advanced point cloud manipulation techniques, TailorGS achieves superior fidelity in garment reconstruction, offering a more accurate and reliable tool for extracting and refining sewing patterns from complex 3D data.

The primary contributions of this thesis are as follows:

- **Development of TailorGS:** Introduction of a novel method that integrates physical simulation and Gaussian Splatting to refine sewing patterns for accurate garment reconstruction.
- **Enhanced Sewing Pattern Extraction:** Demonstration of improved fidelity in sewing pattern extraction from 3D point cloud data, particularly for complex and out-of-distribution garment shapes.
- **Comprehensive Evaluation:** Extensive experimentation showcasing TailorGS’s superior performance compared to existing garment reconstruction methods, validated through both qualitative and quantitative analyses.
- **Robust Methodology:** Provision of a robust and versatile framework that addresses dataset limitations and mathematical complexities inherent in garment reconstruction.
- **Application Demonstrations:** Illustration of TailorGS’s applicability in real-world scenarios, highlighting its potential impact on industries such as fashion design and virtual avatar creation.

Through these contributions, TailorGS represents a significant advancement in the field of garment reconstruction, offering a more accurate and reliable tool for extracting and refining sewing patterns from complex 3D data. This work not only addresses the current limitations faced by existing models but also paves the way for future innovations in digital fashion and virtual garment simulation.

Anywear

Beyond its research objectives, this work serves as the backbone for a larger initiative known as Anywear, a startup dedicated to advancing fashion and virtual try-on technologies. Founded in April 2024 at the Dock3 Incubator Lab with my co-founders Andrea Sanchietti, Martina Attisani, and Omid Ghorbani, Anywear aims to address the significant environmental impact caused by the return of garments. Returned items contribute to environmental damage through transportation fuel emissions and the accumulation of wasted clothing in landfills. Anywear seeks to mitigate this issue by providing users with a reliable preview of how garments fit, both in terms of size accuracy and visual fidelity.

To achieve this, Anywear requires both an accurate avatar representation of the user and a precise 3D model of the garment. While the avatar generation leverages well-established methods based on the SMPL model, which effectively approximates user body shapes from images, garment reconstruction remains a challenge. Existing garment reconstruction methods lack the necessary accuracy for commercial applications, limiting their practicality for virtual try-on solutions. Recognizing this gap, I embarked on this research to develop TailorGS, aiming to enhance garment reconstruction accuracy and thereby empower Anywear to become a leading player in the virtual fashion industry.

This thesis focuses exclusively on the garment reconstruction aspect, providing a research-driven analysis to evaluate the quality and potential of TailorGS. By addressing the inaccuracies of current models and improving the fidelity of sewing pattern extraction, this work contributes directly to the core functionality of Anywear, ensuring that users receive precise and trustworthy garment previews. Through this integration, TailorGS not only advances the academic field of garment reconstruction but also supports the practical goals of reducing environmental waste and enhancing the virtual try-on experience.

Chapter 2

Related Work

Francesco: All the works from the notion, I will try to divide them based on their approach

Francesco: Talking about the fact that most works use sewing patterns and how do they represent and from what

2.1 Sewing Patterns

2.2 Point Cloud

2.3 Generative approaches

2.4 Other approaches (TBD)

2.5 Garments datasets

Francesco: Different Datasets and what information they put. Also put the table with all the information to sum up and compare datasets

Chapter 3

Background

3.1 Sewing Patterns

RM:: Mettere un'immagine Sewing patterns are fundamental to garment construction, serving as the blueprint from which intricate clothing designs are realized. At their core, sewing patterns are collections of two-dimensional (2D) fabric pieces, known as panels, which are meticulously designed and arranged to form the desired three-dimensional (3D) garment when stitched together. These patterns encapsulate the essential structure of a garment, detailing the precise shapes, sizes, and placements of each panel relative to the human body model **RM:: Non mi è chiara questa ultima frase; in che senso "relative to the human body model"?**, typically represented by the Skinned Multi-Person Linear⁽¹³⁾ (SMPL) model in computational applications.

The concept of sewing patterns has evolved significantly since their inception, initially introduced as manual templates used by tailors to cut and assemble fabric into wearable garments **RM:: Hai una citazione su questo? Puoi cercare su scholar filtrando solo per articoli molto vecchi.** Over time, advancements in technology and computational methods have transformed sewing patterns into sophisticated digital constructs that integrate seamlessly with machine learning and computer vision techniques.

In the context of garment reconstruction, sewing patterns offer a structured and interpretable representation that bridges the gap between raw 3D data and coherent garment models. By decomposing a garment into flat panels, sewing patterns simplify the inherently complex nature of 3D garments, making it easier to handle challenges such as data scarcity and occlusions that often plague 3D data processing. This simplification is especially advantageous for machine learning models, which can more effectively learn and generalize from 2D representations compared to their 3D counterparts.

3.1.1 Representation of Sewing Patterns

The representation of sewing patterns is critical for their integration into computational frameworks. According to⁽⁹⁾ **RM:: Forse preferisco le reference non come**

apice, però è gusto personale, se ti piace questo formato si può tenere così., a sewing pattern is defined by a collection of panels and their corresponding stitching information. Each panel within a sewing pattern is characterized by several key attributes:

- **Vertices:** These define the corners of each panel, determining its basic shape and size. Vertices are represented as 2D coordinates that outline the perimeter of the panel.
- **Edges:** The sides of each panel connect the vertices, forming closed polygons. Edges can be either straight lines or curved segments, often represented using Bezier curves to allow for smooth and scalable representations of complex shapes.
- **Stitch Information:** This specifies how the edges of one panel connect to those of another, effectively stitching the garment together digitally. Stitching relations ensure that when panels are joined, the assembled garment maintains structural integrity and aesthetic consistency.
- **Material Properties:** Each panel is associated with material properties that define the type of fabric, its elasticity, thickness, and other relevant attributes. These properties influence the garment's drape and fit, impacting how it behaves when worn or simulated in a virtual environment.
- **Panel Type:** Panels are categorized based on their function and placement within the garment, such as sleeves, collars, bodices, or pant legs. This classification provides a semantic understanding of each component's role within the overall garment structure.

This structure is often formalized in datasets such ⁽⁹⁾ and SewFactory ⁽⁴⁾, where sewing patterns are meticulously annotated with detailed information about each panel's geometry, material properties, and stitching relations.

3.1.2 Mathematical Representation

The mathematical foundation of sewing patterns is integral **RM:: usare la parola "integral" subito dopo "mathematical representation" è un po' pun, cambierei** to their functionality in garment reconstruction. According to ⁽⁹⁾, a sewing pattern is composed of two primary components: a collection of panels and their corresponding stitching information. Formally, a sewing pattern can be described as a set $\mathcal{P} = \{P_i\}_{i=1}^{N_p}$ of N_p panels and a stitching set $\mathcal{S} = \{S_j\}_{j=1}^{N_s}$, where each panel P_i is defined by a closed 2D polygon comprised of N_i edges $\{E_{i,j}\}_{j=1}^{N_i}$. **RM:: Manca definizione di S_j**

Each edge $\{E_{i,j}\}$ within a panel P_i is represented mathematically by a Bezier curve, which can be described using four scalar parameters: x, y, c_x and c_y . Here (x, y) denotes the 2D vector from the start to the end of the edge, while (c_x, c_y) represents the control point of the Bezier curve in a local coordinate system. This parametric representation allows for the precise definition of complex and smooth edges essential

for accurate garment reconstruction.

Furthermore, each panel P_i is associated with a 3D rotation matrix $R_i \in SO(3)$ and a translation vector $T_i \in \mathbb{R}^3$, which describe the relative pose and position of the panel with respect to the underlying human body model, typically represented by SMPL⁽¹³⁾. These transformations are crucial for accurately simulating how the garment drapes and conforms to the body, facilitating realistic and dynamic garment reconstruction.

The stitching information $\mathcal{S}$ **RM::** Non dovrebbe essere S_j ? is defined by pairs of edges $\{(E_{i,j}, E_{i',j'})$ **RM::** Qui l'uso degli indici diventa un po' complicato; forse penserei se togliere l'indice da S_j , o usare altri nomi per gli indici, tipo k, l , indicating that the j -th edge of panel P_i is to be sewn with the j' -th edge of panel $P_{i'}$. This stitching relationship ensures that the assembled garment maintains structural coherence and aesthetic consistency, mirroring real-world sewing practices.

To simulate a diverse range of garment data, sewing patterns incorporate unique material parameters for each garment, corresponding to specific fabric types and textile images. Common fabric presets such as cotton, silk, and velvet are used, each with distinct properties like shininess, elasticity, and texture. Random perturbations are introduced to these properties to simulate variations in fabric behavior, enabling the generation of garments with different levels of drape and fit.

3.1.3 Advantages of Sewing Patterns in Garment Reconstruction

RM:: Sposterei questo in Sezione 3.1 come paragrafo: Prima motiviamo (Cosa sono i SP, a cosa servono, e perché sono vantaggiosi), poi descriviamo come sono fatti. One of the primary advantages of using sewing patterns in garment reconstruction is their ability to simplify the inherently complex nature of 3D garments into manageable 2D representations. By breaking down a garment into flat panels, sewing patterns reduce the dimensionality of the problem, allowing for precise control over the reconstruction of each component, ensuring that the final 3D model accurately reflects the intended design.

Moreover, sewing patterns mitigate common challenges associated with 3D data, such as occlusions and data sparsity. Traditional 3D reconstruction methods often struggle with incomplete or noisy data, leading to fragmented and inaccurate models. In contrast, sewing patterns provide a predefined and structured framework that maintains consistency and continuity across different parts of the garment, even when the underlying 3D data is imperfect. **RM::** Questo paragrafo non mi è chiarissimo: nel senso che i SP sono ottenuti per vestiti molto puliti; inoltre rispetto a una mesh sono discontinui (hai gli stitching di pannelli). Forse direi più: in 3D modifiche a una mesh possono creare artefatti, intersezioni, ecc. Mentre i SP essendo pannelli 2D e realizzati poi attraverso simulazione fisica, possono essere modificati e tengono la garanzia che la realizzazione sia ancora una geometria coerente.

Another significant advantage of sewing patterns is their interpretability and ease of manipulation. Unlike raw 3D point clouds or meshes, sewing patterns offer a

clear and organized structure that closely mirrors the real-world process of garment construction. This alignment facilitates the integration of sewing patterns with machine learning models, enabling automated processing and refinement.

3.1.4 Panel Representation in JSON Format

To facilitate the digital representation and processing of sewing patterns, many modern approaches utilize structured data formats such as JSON. This structured format allows for the precise definition of each panel’s geometry, stitching instructions, material properties, and panel classifications, enabling automated processing and manipulation within digital workflows.

For instance, datasets like Maria Korostoleva’s **RM:: immagino questo vada rimpiazzato con delle citazioni; mettici dei to-do in caso così non ti dimentichi** collection and SewFactory employ a very similar JSON-based pattern templates to encode sewing patterns. These templates include comprehensive details such as the vertices of each panel, their relative positions through translation and rotation matrices, and the connections between panels via stitching information. By standardizing the representation of sewing patterns in JSON, these datasets ensure interoperability and scalability, allowing machine learning models to efficiently parse and utilize the data for training and evaluation purposes.

3.2 SMPL

The Skinned Multi-Person Linear (SMPL) model is a widely adopted parametric 3D human body representation in the fields of computer vision and computer graphics. Introduced by Loper et al.⁽¹³⁾, SMPL provides a realistic and flexible framework for modeling diverse human body shapes and poses. At its core, SMPL uses a set of shape parameters to capture the anatomical variations across different individuals and pose parameters to represent the body’s articulation in various positions. This combination allows SMPL to generate highly accurate and anatomically plausible 3D human meshes that can be easily manipulated for various applications. One of the key strengths of the SMPL model lies in its use of linear blend skinning (LBS) for pose-dependent deformations. LBS enables smooth and natural transitions between different body poses by linearly blending the influence of joint transformations on the mesh vertices. This approach not only ensures realistic movement but also facilitates efficient computation, making SMPL suitable for real-time applications such as virtual try-ons and interactive avatar systems. Additionally, SMPL incorporates a statistical shape space derived from a large dataset of scanned human bodies, allowing it to generalize well to unseen body shapes and poses. Building upon the foundation laid by SMPL, several extensions have been developed to enhance its capabilities. For instance, SMPL-X⁽¹¹⁾ extends the original model by incorporating detailed facial expressions and hand articulations, providing a more comprehensive representation of the human body. This extension is particularly beneficial for applications requiring fine-grained control over facial and hand movements, such as virtual reality (VR) avatars and advanced animation systems. Another notable extension is SMPL+D, which integrates dynamic components to model soft tissue

deformations, further improving the realism of body movements. In the context of garment reconstruction, SMPL serves as an essential backbone for accurately representing the underlying human body onto which garments are draped and simulated. By providing a consistent and detailed body model, SMPL enables precise alignment and fitting of sewing patterns, facilitating the creation of realistic 3D garment models. This alignment is crucial for ensuring that reconstructed garments accurately reflect the intended design and fit, thereby enhancing the overall quality and reliability of virtual try-on systems and digital fashion applications. Moreover, SMPL's compatibility with various machine learning frameworks allows for seamless integration with garment reconstruction algorithms. For example, when combined with sewing pattern-based approaches, SMPL can effectively bridge the gap between 2D pattern representations and 3D garment geometries. **RM::** Forse proverei a fare un po' di spiegazione formale di cosa è SMPL (blend shapes, LBS, ...). Non mi pare che SMPL lo usiamo molto quindi non è importante sia super dettagliato, ma un po' di formalismo forse per background ci sta.

3.3 Warp

In the realm of physical simulations, having tools that can balance performance and precision is essential. Warp⁽⁶⁾ emerges as a robust solution, offering a high-performance GPU-accelerated framework tailored for advanced physics simulations. Developed by NVIDIA, Warp is a versatile Python library designed to handle a multitude of physical phenomena, including fluid dynamics, path tracing, and notably, cloth simulation. While Warp encompasses a broad spectrum of functionalities, this thesis leverages its cloth simulation module to address the intricate challenges of garment modeling.

Warp's approach to cloth simulation is both sophisticated and efficient, primarily due to its optimized handling of collisions and material interactions. At its core, Warp employs Position-Based Dynamics⁽¹⁴⁾ (PBD), a method that ensures numerical stability and real-time performance by iteratively solving constraints to simulate physical interactions. This is particularly advantageous for garment reconstruction, where the delicate interplay between fabric panels must be meticulously managed to achieve realistic draping and movement. Moreover, Warp incorporates advanced collision detection algorithms that prevent fabric from intersecting unnaturally, preserving the integrity of the garment's structure even under complex poses and movements.

Interestingly, Warp's flexibility is demonstrated in the work of Maria Korosteleva **RM::** forse metterei "Korosteleva et al., like ...". Mi sembra che "nome cognome" sia un po' informale, e inoltre sembra che ha fatto tutto lei; ci sono anche gli et al.! lol, the creator of the Neural Tailor framework⁽⁸⁾, Warp is utilized to bridge the gap between sewing pattern outputs and realistic 3D garment simulations⁽⁹⁾. This integration is particularly noteworthy as it aligns perfectly with the JSON format used for sewing pattern specifications, facilitating effortless interoperability between different models such as SewFormer and Neural Tailor.

3.3.1 UV Mapping

RM:: Anche qui, svilupperei un po'. "UV mapping" è un caso di "Surface Parametrization". Ti puoi rifare al libro "Polygon Mesh Processing", capitolo 5 (si trova in PDF, se non lo trovi fammi sapere, dovrei avercelo da qualche parte) Transitioning from the dynamic simulations facilitated by Warp, the concept of UV Mapping becomes essential in bridging the gap between 3D garment models and their 2D sewing patterns. UV Mapping is a technique used to project the surface of a 3D mesh onto a 2D plane, effectively "unwrapping" the object much like peeling an orange and laying its skin flat. This process is fundamental in computer graphics for applying textures to 3D models, but its utility extends profoundly into garment reconstruction.

Francesco: Change example, can create confusion with the real use of uvTo illustrate, imagine attempting to apply a seamless pattern, such as polka dots or stripes, onto a complex 3D garment like a fitted dress. Directly mapping these patterns onto the curved surfaces would result in distortions and misalignments. UV Mapping resolves this by providing a structured and organized framework where each point on the 3D mesh corresponds to a specific location on the 2D plane. This ensures that textures and patterns maintain their intended appearance without distortion, preserving the garment's aesthetic integrity.

In the context of sewing patterns, UV Mapping serves a dual purpose. Firstly, it facilitates the accurate projection of 3D garment geometries back onto their original 2D sewing patterns. This is crucial for refining and adjusting sewing patterns to achieve precise alignment with the reconstructed garment. However, the relationship between UV Mapping and garment reconstruction is not without its complexities. Unlike simple geometric transformations, the mapping between 3D meshes and 2D planes is inherently nonlinear. Slight modifications to the mesh, such as small deformations or alterations in fabric folds, can lead to unpredictable changes in the UV coordinates. This nonlinearity poses challenges for directly manipulating UVs to achieve desired adjustments in sewing patterns. Consequently, while UV Mapping provides invaluable correspondence information between 2D and 3D representations, it necessitates sophisticated techniques to manage and interpret these transformations effectively.

3.4 Technical Works

RM:: Li chiamerei magari: "Sewing pattern regression methods", e li sposterei sopra, dopo i sewing pattern. In chiusura di questa parte, metterei un paragrafo tipo "Limitations" o "remark", dove sintetizzi un attimo qual è il problema: non generalizzano bene ed è difficile da immaginare un data prior sufficiente per coprire la varietà di vestiti, in geometria e texture **Francesco:** Here I will explain briefly how the two methods works (like I did in the ppt since those two works are the backbone of our model

3.4.1 Neural Tailor

3.4.2 SewFormer

Limits of Sewformer and other methods

Francesco: This name will change, but basically I would like to put the experiments that we did to find the limits of sewformer, with all the images we found like v-neck, sleeveless. This section can be putted also in the results chapter instead of here.

3.5 Gaussian Splatting

In the rapidly evolving landscape of computer graphics and computer vision, Gaussian Splatting (GS) has emerged as a transformative technique for rendering and reconstructing complex 3D scenes. Originally introduced to overcome the limitations of traditional 3D reconstruction methods, Gaussian Splatting leverages the power of probabilistic representations to achieve high-fidelity and efficient modeling of intricate geometries. At its core, GS represents 3D scenes using a collection of Gaussian functions, or "splats," each characterized by parameters such as position, covariance, color, and opacity⁽²⁾.

3.5.1 Overview of Gaussian Splatting

The fundamental concept of Gaussian Splatting involves approximating the surface of a 3D object with a dense set of overlapping Gaussian ellipsoids. Each Gaussian splat is defined by its mean vector $\mu \in \mathbb{R}^3$, which specifies the center position in 3D space, and a covariance matrix $\Sigma \in \mathbb{R}^{3 \times 3}$, which determines the shape, size, and orientation of the ellipsoid. Additionally, each splat is associated with color and opacity attributes, enabling the representation of both the geometric and visual properties of the scene. By adjusting these parameters, Gaussian splats can collectively approximate the fine details and smooth surfaces of the target object, providing a flexible and scalable means of modeling complex geometries.

A great way to visualize this is to imagine reconstructing a 3D model of a bicycle from a few images taken from different angles. In a point cloud representation, the bicycle would be depicted by thousands of tiny points, each representing a part of its frame, wheels, and other components. However, the gaps between these points often lead to a lack of smoothness in the reconstruction. With GS, instead of isolated points, each part of the bicycle is covered by Gaussian ellipsoids, which overlap and blend to create a continuous, smooth surface. This provides a much clearer and more accurate depiction of the bicycle, especially when viewed from various angles.

3.5.2 How Gaussian Splatting Works

The Gaussian Splatting pipeline starts by initializing a set of 3D Gaussians based on the input data, which could be a point cloud generated through techniques like Structure from Motion (SfM)⁽¹²⁾ or another form of 3D representation. Each point is assigned a Gaussian distribution that encapsulates its local uncertainty and density.



Figure 3.1 Point Cloud **Figure 3.2** Opacity $\alpha = 0$ **Figure 3.3** With opacity

Figure 3.4 [Francesco: images and caption to be fixed](#)

These Gaussians are then projected onto the 2D image plane through a differentiable rasterization process. During this operation, each Gaussian is "splatted" onto the image, creating smooth approximations of the scene's surface as viewed from different angles. Because the projection is differentiable, gradient information flows through the system, allowing the model to adjust and improve the representation of each Gaussian. Gaussian Splatting iteratively refines the 3D Gaussian parameters by adjusting their size, orientation, and opacity, ensuring that each part of the scene is represented accurately.

Key to this process is adaptive density control, a technique that adjusts the size and opacity of each Gaussian splat depending on its importance in the scene and on the level of detail required. This feature has the ability to clone or split Gaussians dynamically during the optimization process. In regions where the initial Gaussians under-represent the geometry, new Gaussians are cloned to increase local density and enhance detail (ref to figure). Conversely, in areas of over-reconstruction, Gaussians can be split to reduce redundancy and optimize the representation (ref to figure). For example, areas with fine details, such as the spokes of a bicycle wheel, will have smaller, denser Gaussians to capture their structure accurately. In contrast, larger, simpler areas, like the bicycle frame, can be represented by fewer, larger Gaussians.

3.5.3 Mathematical Representation

Mathematically, Gaussian Splatting relies on the properties of multivariate Gaussian distributions. The probability density function (PDF) of a Gaussian splat at a point $x \in \mathbb{R}^3$ is defined as:

$$G(x) = \frac{1}{(2\pi)^{3/2} |\Sigma|^{1/2}} \cdot \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right) \quad (3.1)$$

This formulation allows each splat to contribute to the overall representation based on its proximity and orientation relative to the target point. When multiple splats overlap, their combined effect creates a smooth and continuous surface that closely approximates the desired geometry, effectively bridging gaps and mitigating noise inherent in raw 3D data⁽²⁾.

3.5.4 Advantages of Gaussian Splatting

RM:: anche in questo caso, metterei tutta la discussione dei "pro" prima della formalizzazione matematica. Compared to other rendering and reconstruction methods such as Neural Radiance Fields (NeRFs)^{(10) (7) (3) (5)}, Gaussian Splatting offers distinct advantages in terms of efficiency and adaptability. While NeRFs represent scenes through volumetric grids that require extensive computational resources and memory, GS provides a more compact and continuous representation by modeling surfaces with Gaussian ellipsoids. One of the significant advantages of Gaussian Splatting is its ability to handle incomplete or noisy data. In many real-world scenarios, such as garment scanning or 3D object reconstruction, the input data may be occluded or contain gaps. Point cloud-based methods often struggle with this, as missing data can lead to holes in the final model. However, Gaussian Splatting's probabilistic approach allows it to "fill in the gaps" by blending neighboring Gaussians, resulting in smooth, continuous reconstructions even when the data is sparse.

3.5.5 Applications in Garments

Francesco: I don't know if I will put this paragraph, I can say that there is no work that uses gaussian splitting in the garment word (sewing patterns), GS is used in retargeting and other similar tasks

3.5.6 Challenges and Limitations

Despite its many strengths, Gaussian Splatting is not without challenges. One of the primary difficulties lies in optimizing the parameters of the Gaussians. The model must balance the trade-off between capturing fine details and avoiding overfitting, which can result in overly complex or redundant Gaussians. Additionally, while Gaussian Splatting excels at capturing smooth, continuous surfaces, it can struggle with high-frequency details or sharp edges that are better suited to traditional mesh-based approaches. **RM::** In generale direi che estrarre e ottenere superfici da GS è un problema di ricerca aperto. Visto che GS si preoccupa solo di rappresentare l'appearance di quello che hai davanti, non ha nessun vincolo sulla geometria. Inoltre ricostruzione da poche viste è ambiguo di per se: la stessa appearance può essere ottenuta da diverse combinazioni di geometria+texture.

Furthermore, Gaussian Splatting's reliance on GPU resources can be a bottleneck, especially when dealing with large, highly detailed scenes. The adaptive density control helps mitigate some of these computational demands, but optimizing the system for different types of data remains an active area of research.

Chapter 4

Methodology

The methodology section delineates the systematic approach adopted to enhance garment reconstruction through the refinement of sewing patterns using Gaussian Splatting. The foundational premise of this research is grounded on the observation that contemporary state-of-the-art models frequently fail to accurately predict sewing pattern configurations, especially when addressing nuanced details such as neck curvature or complex structures like single-shoulder dresses. These inaccuracies can lead to significant deviations from the intended garment design, thereby compromising the fidelity of the reconstructed 3D models. TailorGS addresses these shortcomings by meticulously identifying the failure points of existing models and refining the sewing patterns based on reference point clouds or meshes. This refinement process ensures that the final sewing patterns are both precise and aligned with the expected garment structure, thereby enhancing the overall quality and reliability of garment reconstruction.

Current models often misinterpret certain aspects of sewing patterns, resulting in errors ranging from subtle deviations to major structural inaccuracies. For instance, a model might incorrectly predict a two-shoulder dress when the actual garment has only one shoulder, or it might fail to capture the precise curvature of a V-neckline. The goal of TailorGS is to bridge the gap between the predicted sewing patterns and the correct garment design through a process of refinement based on reference garment data. By leveraging Gaussian Splatting, TailorGS systematically adjusts the sewing patterns to better align with the ground truth, ensuring that both minor and major discrepancies are effectively corrected.

This section breaks down the methodological pipeline, detailing each stage of the process—from nomenclature definitions, problem formulation, and baseline methods to the more advanced stages of refinement and optimization. Each component is designed to gradually refine sewing patterns, ensuring they align with a reference garment model.

4.0.1 Nomenclature

To ensure clarity and consistency throughout this methodology, it is essential to establish a clear nomenclature. The following key terms are defined to facilitate

understanding **RM::** Forse darei un simbolo a ogni definizione, per semplificare i riferimenti nel testo e scrivere alcuni passaggi in forma di equazioni (anche se vorrai includere una pipeline figure, è più facile annotarla):

- **GT Garment Mesh (or Point Cloud) $\mathcal{G}$:** This refers to the ground truth 3D representation of the garment, either in the form of a mesh or a point cloud, which serves as the reference for the refining process.
- **Sewing Pattern Specifications:** These are the detailed parameters and attributes predicted by the backbone models (such as SewFormer or Neural-Tailor), encoded in a JSON format, which define the structure and properties of the sewing patterns.
- **GT Sewing Pattern Specifications:** The precise parameters and attributes of the ground truth sewing pattern, providing a detailed reference for evaluating the accuracy of the predicted sewing patterns.
- **Sewing Pattern:** An image representation of the garment's sewing pattern, visualizing the 2D panels and their arrangement before assembly into a 3D model.
- **GT Sewing Pattern:** The ground truth sewing pattern, which represents the expected 2D sewing pattern layout corresponding to the GT Garment Mesh.
- **GT Panels Segmentation:** A text file annotation that specifies the segmentation of the garment into distinct panels, detailing how each panel corresponds to specific sections of the garment.

These terms are visually represented in an accompanying figure **Francesco:** (Figure 1), which illustrates the relationship between the GT Garment Mesh, sewing patterns, and their specifications. This nomenclature serves as a foundational reference, ensuring that readers can easily navigate the subsequent sections of this methodology **Francesco:** I will like to say that the complete nomenclature or "glossary" is at the end or in the beginning where I will put everything (here a lot of things are missing in the list).

4.0.2 Problem formulation

The core problem addressed in this research can be formally defined as follows: Given a 3D mesh or point cloud representation of a garment $\mathcal{G}$ (where $\mathcal{G}$ denotes either the mesh or point cloud), the objective is to accurately reconstruct the garment's sewing patterns $\mathcal{P} = \{P_i, S_i\}_{i=1}^N$. Here, a sewing pattern comprises a set of panels and their corresponding stitching information, structured as $P_i = \{V, E, R, T\}$. In this notation, V represents the vertices, which are the 2D coordinates defining the corners of each panel, and E denotes the edges connecting these vertices, which can be either straight lines or curves. R corresponds to the rotation matrix, describing the orientation of the panel relative to the standard SMPL model, while T is the translation vector that indicates the position of the panel with respect to the SMPL model.

Additionally, the stitching information S_i consists of pairs of edges $S = \{(i, j), (k, l)\}$, indicating that the j -th edge of panel P_i is sewn to the l -th edge of panel P_k . This formalization captures the spatial and relational attributes necessary for reconstructing the garment's structure accurately.

Mathematically **RM::** invece che Matematicamente direi "more compactly", o "formally", anche se è più una forma compatta di quanto hai scritto sopra, senza troppa mateamtica, the problem can be expressed as:

$$\text{Given } \mathcal{G}, \text{ find } \mathcal{P} = \{P_i, S_i\}_{i=1}^N \text{ s.t. } P_i = \{V, E, R, T\} \text{ and } S_i = \{(i, j), (k, l)\} \quad (4.1)$$

This formulation encapsulates the essence of the garment reconstruction task, highlighting the need for precise mapping between the 3D garment representation and its corresponding 2D sewing patterns. By addressing this problem, TailorGS aims to bridge the gap between raw 3D data and accurate, interpretable sewing patterns, thereby enhancing the overall quality of garment reconstruction.

4.0.3 Pipeline Overview

The baseline approach serves as the foundational framework upon which TailorGS builds its refinement capabilities. The primary objective of the baseline is to establish a correspondence between the predicted sewing patterns and the ground truth garment mesh. This correspondence is crucial, as it allows for the identification of specific areas—whether it's adding, removing, or adjusting points in the pattern—that need refinement. For instance, consider a garment with a v-neckline. If the predicted sewing pattern represents the neckline inaccurately, the refinement process would adjust the curvature of the neckline by modifying the corresponding points in the sewing pattern, making it sharper or smoother as necessary to align with the ground truth.

The pipeline can be broken down into three main stages: **RM::** forse invece di fare elenco puntato farei paragrafi

- **Sewing Pattern Reconstruction:**

The initial sewing patterns are generated by backbone models such as SewFormer or NeuralTailor. These models output sewing pattern specifications

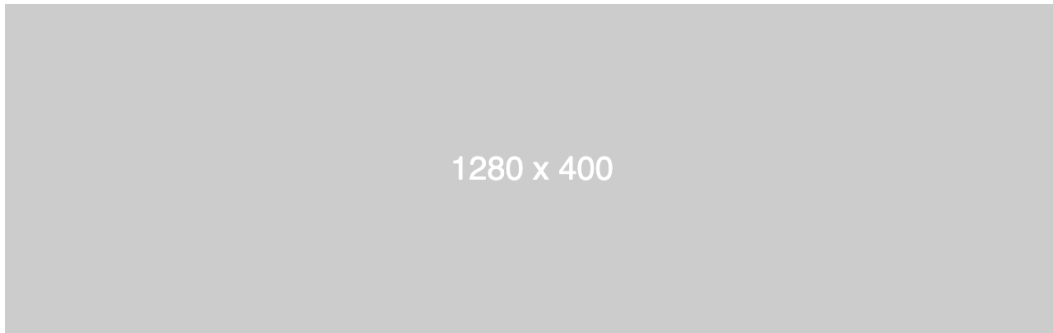


Figure 4.1 Pipeline Architecture image

in a JSON format, detailing the geometry and stitching information of each panel. Using these specifications, the sewing patterns are reconstructed into a 3D environment by mapping each panel’s rotation and translation relative to the standard SMPL model. This process transforms the 2D sewing patterns into a preliminary 3D garment model **RM::** Magarei anche questo lo definirei quando dai i nomi.

- **Refinement in GS:**

Leveraging the capabilities of Gaussian Splatting, the preliminary 3D garment model undergoes refinement to better align with the GT Garment Mesh. Gaussian Splatting is employed to adjust the positions of sampled points on the garment mesh, enhancing the model’s fidelity to the reference data. This refinement involves dynamically adding, removing, or shifting Gaussian splats based on their alignment with the GT mesh, effectively correcting errors in the initial sewing pattern predictions.

- **Correspondence Mapping:**

Once the garment mesh has been refined, the modifications made in the 3D space need to be accurately reflected back in the 2D sewing patterns. This is achieved through UV Projection, where the refined 3D adjustments are mapped back onto the 2D sewing patterns using the garment mesh’s UV map. This ensures that the final sewing patterns are not only accurate in 3D space but also maintain their structural integrity when represented in 2D.

To visualize the baseline approach, imagine a scenario where the SewFormer model predicts a two-shoulder dress, but the ground truth garment is a one-shoulder design. The Sewing Pattern Reconstruction step generates the initial 3D model with two shoulders. The GS Refiner identifies the excess shoulder splats and removes them, refining the mesh to align with the one-shoulder design. Finally, the Correspondence Mapping step updates the 2D sewing patterns to match the refined 3D garment, ensuring consistency between the two representations.

This baseline establishes a robust framework for garment reconstruction, highlighting the critical role of Gaussian Splatting in refining sewing patterns, bringing them closer to the ground truth garment’s shape and fit. By creating a clear correspondence between 3D garment meshes and 2D sewing patterns, the baseline sets the stage for

TailorGS to further enhance the accuracy and reliability of garment reconstructions. The Gaussian Splatting process serves as a bridge between 3D garment data and 2D sewing patterns, ensuring that the final result accurately reflects the garment’s real-world geometry.

4.1 Sewing Pattern Reconstruction

The process of reconstructing sewing patterns involves setting up a Gaussian Splatting environment, which requires simulating a Structure from Motion (SfM) pipeline to generate accurate image data from multiple viewpoints. In SfM, multiple images of an object are taken from different angles, and the information from these images is combined to reconstruct the 3D geometry of the object. Similarly, in Gaussian Splatting, images of the object (in this case, sewing patterns) are used to optimize the splats, refining their positions and properties to closely align with the ground truth geometry.

However, the challenge lies in the fact that these images alone do not provide sufficient information for this optimization. The position and properties of the cameras capturing these images—such as their location in space, focal length, and field of view—are equally important. For example, imagine capturing an image of the garment from a certain angle. Without knowing the camera’s position or its focal length, it is impossible to determine how this image corresponds to the Gaussian Splatting model, making the reconstruction unreliable. By surrounding the garment with multiple cameras, capturing images from all angles, and storing key camera parameters, it ensures that the Gaussian Splatting algorithm has all the necessary data to optimize the garment’s 3D reconstruction.

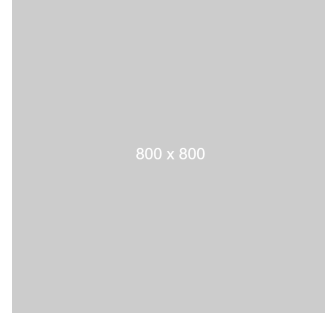
4.1.1 Setting up Gaussian Splatting

To effectively utilize Gaussian Splatting for sewing pattern reconstruction, it is essential to simulate an SfM setup. This involves preparing a 3D environment where virtual cameras are strategically positioned around the garment mesh centered within the scene, serving as a consistent reference point for all subsequent operations, and capturing images from various angles. This setup mirrors how an object might be placed on a stand, with a photographer circling it to capture photos from every angle. A similar concept is applied in photogrammetry, a technique used to create 3D models from 2D images.

By capturing images from diverse angles, Gaussian Splatting can effectively optimize the positions and properties of Gaussian splats to better fit the garment’s geometry. This setup not only provides the necessary visual data but also incorporates critical camera information, such as position, orientation, and intrinsic parameters, which are essential for accurate image projection and loss computation. Moreover, the strategic placement of cameras ensures comprehensive coverage of the garment’s geometry, minimizing blind spots and capturing intricate details from every possible perspective. The following section delves into the specifics of camera setup, which is a cornerstone of the Gaussian Splatting reconstruction process.

Camera

As previously mentioned, the accurate positioning and configuration of cameras are essential for the success of Gaussian Splatting in garment reconstruction. Since the environment is simulated, full control over the camera parameters is enabled, allowing for manual selection and precise adjustments to optimize the capture process for specific requirements. At its core, a camera is defined as an object responsible for capturing images of the garment from specific viewpoints.



The camera model encompasses several attributes that collectively determine how the garment is rendered in each image. These attributes include:

- **Identifier (uid):** Each camera is assigned a unique identifier to distinguish it from others within the environment. This identifier ensures that the images captured from each camera can be accurately traced back to their respective camera settings in the optimization process.
- **Rotation and Translation (R,T):** The orientation and position of each camera relative to the garment are defined by rotation matrices $\mathbf{R}$ and translation vectors $\mathbf{T}$. Mathematically, the camera's orientation can be represented using a rotation matrix $R \in SO(3)$, and its position relative to the origin of the scene is described by a translation vector $T \in \mathbb{R}^3$.

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \quad (4.2)$$

The rotation matrix determines how the camera is angled, while the translation matrix specifies its position in 3D space. These transformations dictate how the garment is viewed from each camera, influencing the angles and perspectives captured in the images. The transformation of a point x in the garment mesh to the camera's coordinate system can be expressed as:

$$x' = R \cdot x + T \quad (4.3)$$

where x' is the transformed point in the camera's coordinate system.

- **Near and Far Clip (z_{near}, z_{far}):** These parameters define the depth range within which objects are visible to the camera. Objects outside this range are not directly rendered in the final image, though they might still influence light bounces and reflections. This is essential in controlling how much of the scene is considered relevant to the reconstruction process. For example, if the near clip is too far from the garment, important close-up details may be lost, while setting the far clip too close might exclude significant parts of the garment in the background.
- **Focal Length/Field of View (FOV):** The focal length f controls the amount of zoom in the image and affects how much of the scene is visible.

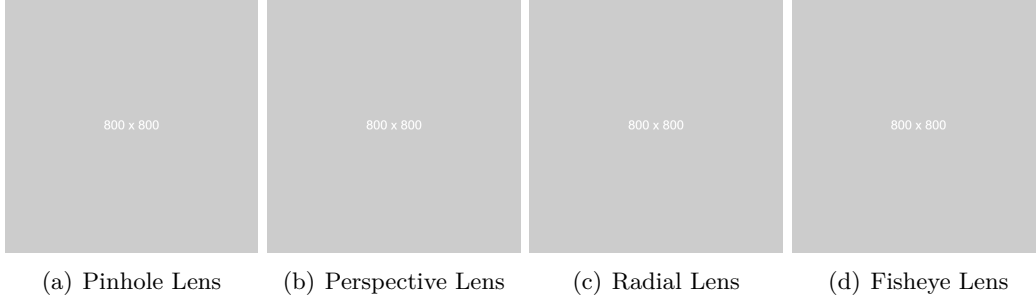


Figure 4.2 TBD

It is the distance required by a lens to focus parallel rays of light onto the camera’s image plane. Conversely, shorter focal lengths provide a wider view of the scene. The field of view (FOV) represents the angular extent of the scene visible to the camera. These two parameters are inversely related and the relationship between focal length and FOV is given by:

$$FOV = 2 \cdot \arctan \frac{d}{2f} \quad (4.4)$$

where d is the sensor size. A longer focal length results in a narrower FOV (more zoom), while a shorter focal length provides a wider FOV (less zoom). Selecting appropriate focal lengths is crucial for capturing both detailed and holistic views of the garment without introducing significant distortion.

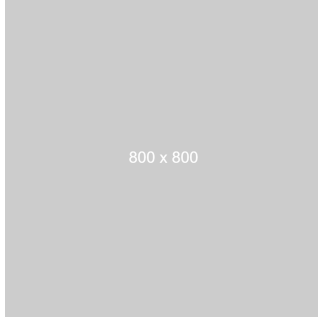
- **Lens Type:** The type of lens used in the camera significantly alters the resulting image. Different lenses introduce varying levels of distortion, which must be accounted for in the Gaussian Splatting process. Common lens types include:
 - **Pinhole Lens:** A simple lens model that projects the scene onto the image plane without introducing distortion. It assumes that all rays passing through the lens converge at a single point, making it a suitable choice for scenarios requiring high-fidelity imaging.
 - **Perspective Lens:** A more realistic model that incorporates some distortion, where objects further from the camera appear smaller. This model is widely used in 3D rendering as it mimics real-world camera behavior.
 - **Radial Lens:** A camera model that introduces radial distortion, causing straight lines to appear curved, particularly towards the edges of the image.
 - **Fisheye Lens:** Provides an extremely wide FOV, capturing nearly 180 degrees of the scene. While useful for certain applications, fisheye lenses can significantly distort the image, requiring careful calibration to mitigate inaccuracies.

Each of these lens types alters the captured image in different ways, understanding and selecting the appropriate lens type is essential for minimizing

distortions that could adversely affect the Gaussian Splatting optimization process. For instance, using a fisheye lens would introduce significant curvature in the captured images, complicating the alignment and refinement of Gaussian splats with the ground truth geometry.

Orbit Camera

To maintain simplicity and ensure a balanced distribution of viewpoints, orbit cameras are employed. Orbit cameras are positioned around the garment in a spherical arrangement, ensuring consistent coverage from all angles with each camera always pointing towards the center of the scene where the garment is placed. Instead



of manual rotation and translation settings, these cameras follow a polar coordinate system characterized by two parameters, azimuth and altitude angles, to define each camera's position on the sphere. The azimuth angle (ϕ) determines the horizontal rotation around the garment, while the altitude angle (θ) defines the vertical position above or below the equatorial plane (**ref to image ???**). In this setup, the garment is placed at the center, and the cameras are positioned on a virtual sphere with a radius r .

The mathematical relationship for positioning an orbit camera on a sphere of radius r using azimuth ϕ and altitude θ is given by:

$$x = r \cdot \cos(\theta) \cdot (\cos \phi) \quad (4.5)$$

$$y = r \cdot \cos(\theta) \cdot (\sin \phi) \quad (4.6)$$

$$z = r \cdot \sin(\theta) \quad (4.7)$$

Where (x, y, z) are the Cartesian coordinates of the camera in the 3D space.

To strategically position the cameras, a sinusoidal pattern is employed, ensuring an even distribution across different elevations and azimuths. For instance, with a set of altitudes $\theta = [-30^\circ, 0^\circ, +30^\circ]$ combined with evenly spaced azimuth angles (i.e. $\phi = [0, 20, 40, \dots, 340]$) a total of 20 orbit cameras can be generated. This balanced distribution captures the garment's full geometry without redundancy, optimizing computational resources while maintaining high reconstruction accuracy.



Figure 4.3 TBD: camera dispo

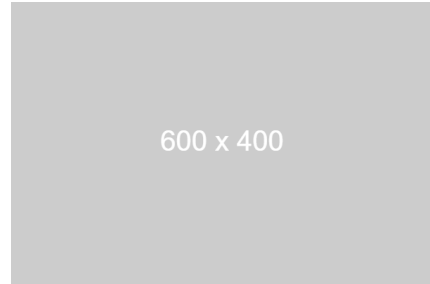


Figure 4.4 TBD

Capturing Images

Once the cameras are strategically positioned, the garment mesh is normalized to fit within the unit sphere, ensuring consistency across all camera views. Mathematically, each vertex v in the mesh is transformed as follows:

$$v_{normalized} = \frac{v}{\|v\|_{\infty}} \quad (4.8)$$

Where $\|v\|_{\infty}$ denotes the maximum absolute value among the vertex coordinates, ensuring that the garment fits neatly within the camera's field of view.

Once the cameras are positioned, they capture images from various viewpoints. These images, combined with their camera metadata, serve as inputs to the Gaussian Splatting algorithm. It is important to note that Gaussian Splatting does not directly utilize the mesh within its environment. Instead, it relies solely on the 2D images captured by the cameras. The process (more in detail in section 4.1.3) begins with an empty scene where Gaussian splats are initialized as a random point cloud within the 3D space. These splats are then optimized through Gaussian Splatting, which adjusts their positions, sizes, and orientations based on the reference images to accurately reconstruct the garment's geometry. This optimization is guided by the computed L1 loss between the rendered images from the Gaussian Splatting model and the ground truth images captured by the cameras.



Figure 4.5 TBD: Figures with some renders all around object

4.1.2 Sewing Pattern Mesh Extraction

In the previous section, it was explained that a mesh will be used to reconstruct and prepare the Gaussian Splatting. However, there is no direct mesh available in this stage, but rather a JSON file containing all the necessary specifications.

The extraction of an accurate mesh from these specifications, although seemingly straightforward, poses several challenges, primarily due to the way the sewing pattern is represented. The procedure is inherently linear: the JSON file provides information about the vertices as 2D points in space, which are then used to reconstruct the final 3D mesh. Initially, curve information is omitted to simplify the mesh creation, focusing solely on the connections between vertices. The `trimesh` library is adopted for mesh construction, where mesh faces are formed by grouping three consecutive vertices, resulting in a basic mesh representation of the sewing pattern. This first iteration creates a simple polygonal representation of the sewing pattern's 2D mesh, which looks like a rough outline of the garment, serving as an initial approximation of the sewing patterns.



Figure 4.6 TBD

Once the 2D mesh is established, the rotation and translation vectors from the JSON file are applied to position the sewing pattern in 3D space. Specifically, each panel's 2D vertices are augmented with a third dimension $z = 0$, essentially transforming them into a flat 3D plane. Then, the 3D position of each panel is achieved by transforming those 3D vertices using an $SO(3)$ rotation matrix, derived from the Euler angles provided in the specification file. The transformation is computed using the following formula:

$$R = \begin{pmatrix} \cos \theta \cos \psi & -\cos \phi \sin \psi + \sin \phi \sin \theta \cos \psi & \sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi \\ \cos \theta \sin \psi & \cos \phi \cos \psi + \sin \phi \sin \theta \sin \psi & -\sin \phi \cos \psi + \cos \phi \sin \theta \sin \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{pmatrix} \quad (4.9)$$

Where θ , ϕ , and ψ are the Euler angles representing the rotation about the x , y , and z axes, respectively. The translation vector is then applied to correctly position the mesh relative to the reference human body model (e.g., SMPL model). The result is a 3D view of the sewing pattern, as shown in the following figure.



Figure 4.7 TBD

However, the introduction of curves complicates the mesh extraction process, necessitating additional steps to accurately represent curved edges. The following subsections delve into the methodologies employed to address these challenges, ensuring that the reconstructed mesh faithfully represents the intended garment design.



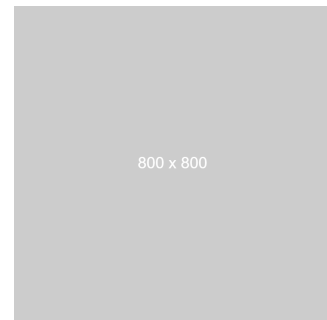
Figure 4.8 TBD: oldvert



Figure 4.9 TBD: new vert

Bézier Curves

A significant challenge in the extraction process arises from accurately representing curved edges. These curves define the contour and fit of a garment, and omitting them would result in an inaccurate or overly simplified representation. The JSON specification file includes information about the curvature of each edge using quadratic Bézier curves, characterized by control points relative to the start vertex and scaled by the norm of the connecting vector.



Specifically, each curve is defined by coordinates c_x and c_y , which determine the curvature's direction and magnitude relative to the straight line between two vertices.

For example, a straight edge between vertices v_1 and v_2 is represented by $c_x = 0$ and $c_y = 0$, indicating no curvature. Conversely, a curved edge with $v_1 \rightarrow v_2 = (0.1, 0.2)$ implies that the control points are offset by $0.1 \times \|v_2 - v_1\|$ and $0.2 \times \|v_2 - v_1\|$ along

the x and y axes, respectively.

$$c_x = 0.1 \cdot \|v_2 - v_1\| \quad (4.10)$$

$$c_y = 0.2 \cdot \|v_2 - v_1\| \quad (4.11)$$

This information allows for interpolation of points along the curve using a standard Bézier equation. For a quadratic Bézier curve defined by points P_0 (starting point), $P_1 = (c_x, c_y)$ (control point), and P_2 (ending point), the curve can be described parametrically as:

$$B(t) = (1 - t)^2 P_0 + 2(1 - t)t P_1 + t^2 P_2 \quad \text{for } t \in [0, 1] \quad (4.12)$$

By sampling the curve additional points are generated along the curve, which are then added to the mesh as new vertices. This sampling process involves interpolating points at every Δs step along the curve, effectively translating the 2D curvature information into discrete vertices that can be incorporated into the mesh. This interpolation results in a dense set of perimetral points that closely follow the curved edges of the sewing pattern (as illustrated in the figure ??), providing the necessary detail for subsequent mesh triangulation.



Figure 4.10 TBD: image of the perimetral points

Mesh Triangulation

Once the perimetral points, including those sampled from Bezier curves, are established, the next step is to construct faces for the mesh. Without the curves, triangulating the mesh was straightforward—simply connecting adjacent vertices (Fig. ??)—but after introducing Bézier curves, the task becomes more complex. Standard triangulation methods, which typically connect three consecutive vertices to form triangular faces, may fall short in accurately representing complex sewing patterns, especially those with internal vertices and holes. For instance, points that are near one another but should not be connected—such as those on opposite sides of a neckline—might mistakenly form a triangle, creating visual and structural errors in the mesh.

To address this, the Delaunay Triangulation algorithm⁽¹⁶⁾ is employed. Delaunay triangulation optimizes the placement of triangles to avoid thin or stretched triangles, ensuring that triangles are formed based on a set of criteria that minimize the likelihood of erroneous connections. However, Delaunay triangulation on its

own may still result in invalid connections, particularly near areas with complex geometry, such as holes or concave regions of the garment.

To overcome this limitation, Constrained Delaunay Triangulation⁽¹⁵⁾ (CDT) is utilized. CDT incorporates boundary constraints to the triangulation that dictate how certain edges should be preserved during triangulation. By defining these constraints based on the perimeter of the sewing pattern and the locations of internal vertices, CDT ensures that the resulting mesh accurately reflects the garment’s intended structure without introducing unwanted connections.

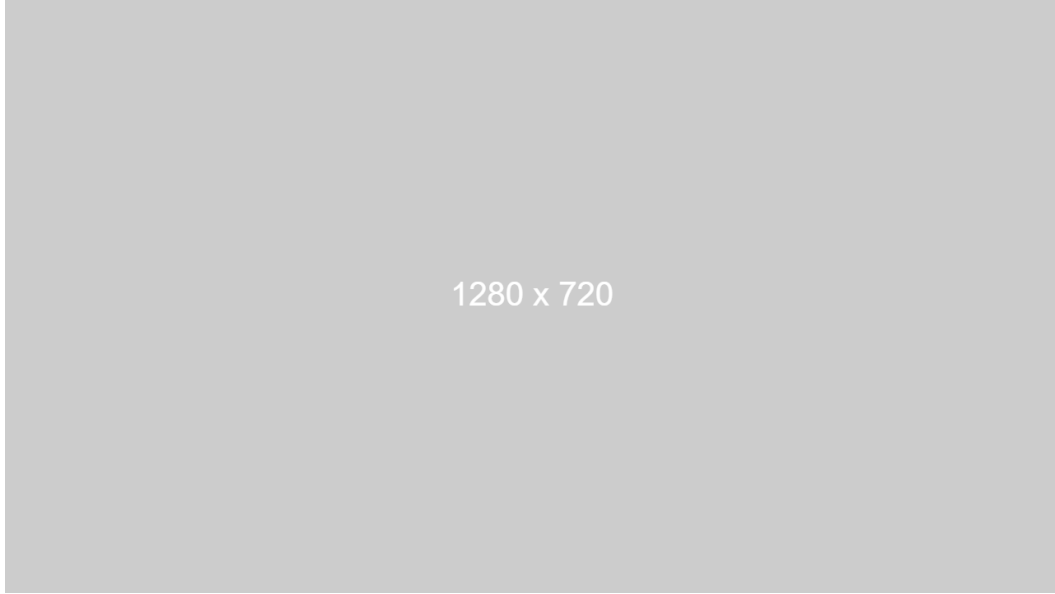


Figure 4.11 TBD: image of the final meshes

4.1.3 The Gaussian Splatting Reconstruction

Once the initial setup is complete, the Gaussian Splatting reconstruction process begins. Initially, random Gaussian splats are placed inside a unit sphere, and the same virtual cameras configured in the previous steps are positioned around the scene to capture the reference images (denoted as `image_cond`) necessary for guiding the reconstruction.

The training process is iterative, where, at each step, a subset of these virtual cameras is selected. A render is produced for each camera at a particular timestamp t , resulting in a generated image x_t , which reflects the current state of the Gaussian configuration. This generated image is then compared to the corresponding reference image $\hat{x}_t$ from the camera. The discrepancy between these two images is quantified using an L1 loss function, which calculates the pixel-wise absolute differences between the generated and reference images. The loss value is then backpropagated through the network, updating the parameters of the Gaussians, which in turn adjust their position, size, and opacity to minimize the difference. As this iterative process

continues, the Gaussians slowly converge to match the underlying geometry of the reference mesh. The progressive refinement of the Gaussians can be observed in the figures below (figure 4.19, where the Gaussians gradually move and align themselves to represent the sewing pattern more accurately. By the final steps of the process, the Gaussians have optimized their positions, forming a precise 3D representation of the mesh.

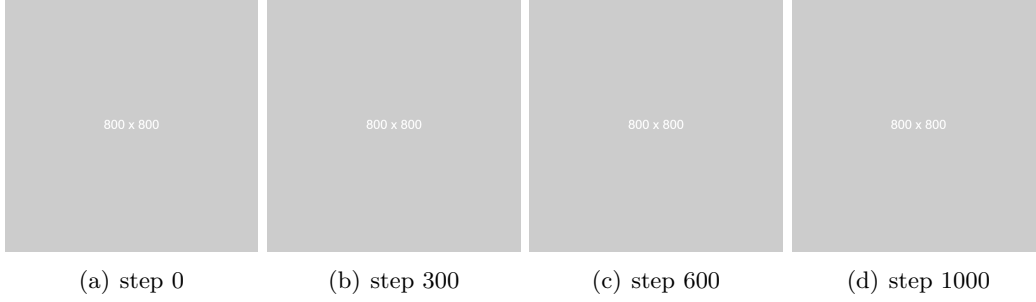


Figure 4.12 TBD

Point Cloud Initialization

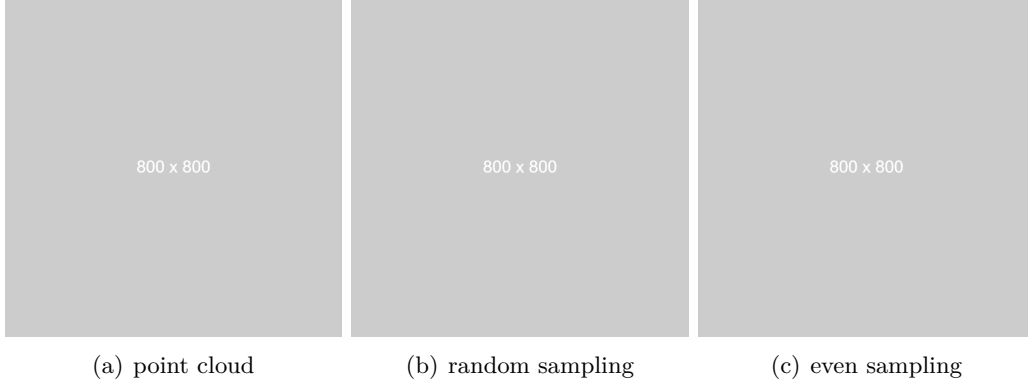
An important improvement in Gaussian Splatting is the initialization of the point cloud. Traditionally, GS begins with a randomly distributed set of Gaussian splats within a predefined space. However, leveraging prior knowledge about the garment’s geometry can significantly enhance the efficiency and accuracy of the reconstruction. Instead of starting with a random point cloud, it is possible to sample points directly from the garment’s mesh (Figure 4.13), providing a more informed starting point for optimization.

By initializing the Gaussian splats with points sampled from the mesh, the algorithm gains a head start in approximating the garment’s structure. This approach reduces the computational complexity, as the splats are already positioned in regions of interest, minimizing the need for extensive movement and adjustment during training. Moreover, starting with a mesh-derived point cloud facilitates better tracking of Gaussian movements, as splats corresponding to specific garment features (e.g., sleeves, collars) remain clustered around their respective areas throughout the refinement process.

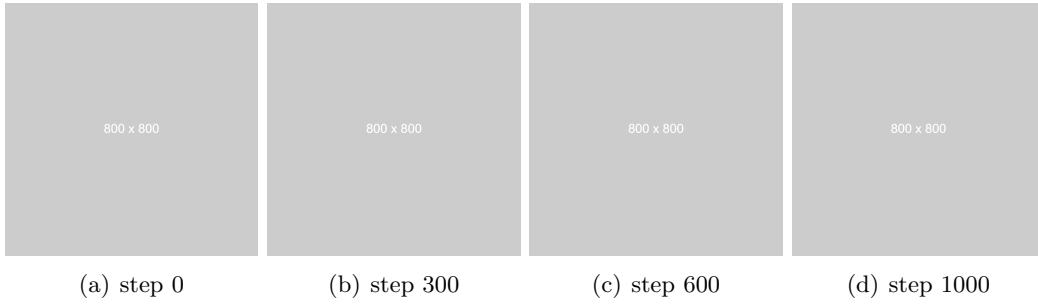
Mathematically, the initialization can be described as follows. Let MMM be the set of vertices in the garment mesh, and Θ be the set of Gaussian splats. Instead of initializing Θ as a random distribution, they are defined:

$$\Theta = \{\theta_i \mid \theta_i \in M\} \quad (4.13)$$

where each θ_i is a Gaussian splat positioned at vertex v_i in the mesh M . This initialization ensures that the splats are distributed in a manner that closely mirrors the garment’s geometry.

**Figure 4.13** TBD

The benefits of mesh-sampled initialization are evident in the optimization trajectory. As depicted in Figure 4.14, starting with a mesh-sampled point cloud leads to a more rapid convergence towards the target garment geometry compared to a random initialization. This efficiency is particularly advantageous when dealing with complex garments that require precise representation of intricate details. Furthermore, this approach enhances the interpretability of the reconstruction process. By maintaining a correspondence between the initial splats and the garment’s mesh vertices, it becomes easier to analyze how specific regions of the garment are being refined.

**Figure 4.14** TBD: GS steps with point cloud

Random Background

Another significant consideration in the Gaussian Splatting reconstruction process is the handling of the background in rendered images. A static background can inadvertently lead to the formation of artifacts, as the algorithm may introduce splats that mimic background elements rather than the garment itself.

For instance, if a static white background is used consistently across all rendered images, GS may interpret this as a feature to be replicated, resulting in unintended white splats scattered throughout the scene. These artifacts can distort the garment’s appearance, making the reconstruction inaccurate despite achieving low L1 loss values. To mitigate this issue, introducing variability in the background is essential.

One effective strategy is to employ a Random Background during the rendering process. By alternating between different background colors or patterns, GS is discouraged from associating specific background features with the garment. This randomness ensures that the optimization focuses solely on the garment’s geometry, as the background no longer presents consistent features that could be mistakenly replicated by the splats.

The implementation of a random background can be achieved by alternating between predefined background colors (figure ??) or generating dynamic patterns during each training iteration. For example, a simple binary approach involves switching between black and white backgrounds: $B = \{\text{Black}, \text{White}\}$. At each step t , the background b_t is selected uniformly:

$$\begin{cases} \text{Black,} & \text{with probability 0.5} \\ \text{White,} & \text{with probability 0.5} \end{cases} \quad (4.14)$$

This binary choice introduces sufficient variability to prevent GS from overfitting to a specific background, thereby reducing the likelihood of artifact formation.

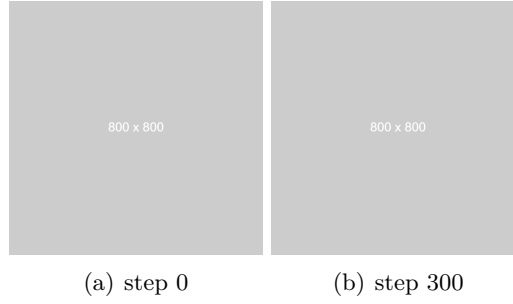


Figure 4.15 TBD: same render with different backgrounds

The effectiveness of random backgrounds is illustrated in Figure 4.16. When a static background is used, Gaussian splats begin to cluster in ways that replicate the background color (fig a). However, by introducing randomness in the background, these unintended splats are mitigated, allowing the algorithm to focus exclusively on the garment’s geometry, resulting in a cleaner and more accurate reconstruction (fig b).

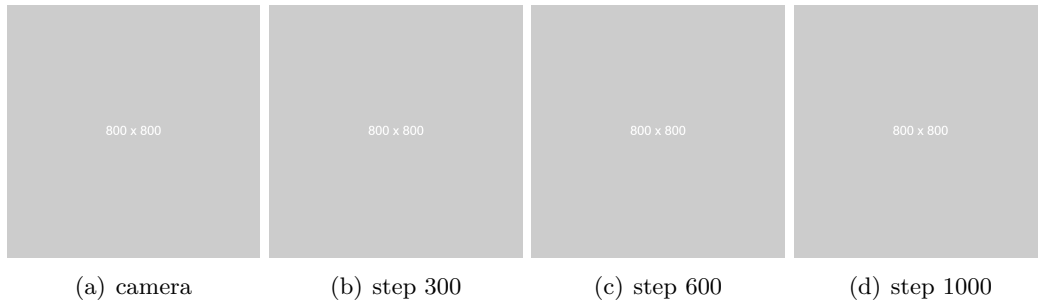


Figure 4.16 TBD: example of a single color background

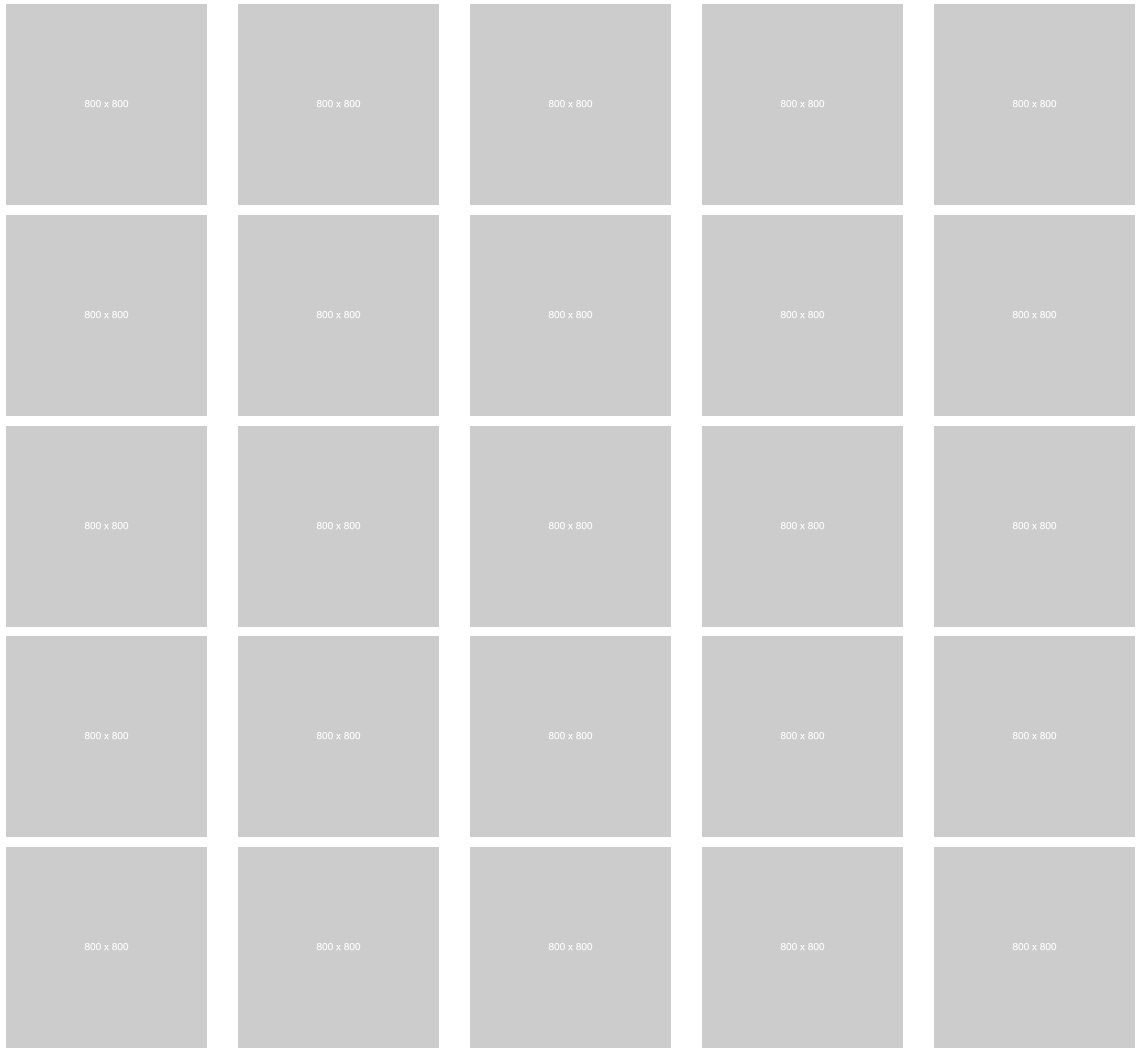


Figure 4.17 Input Images

4.1.4 Towards the Garment Mesh

Before proceeding to the second step of the pipeline, which focuses on refining the output, it is essential to address a critical adjustment in the pipeline approach. Initially, the refining component was designed to create a direct correspondence between the sewing patterns and the ground truth (GT) garment mesh. **Francesco:** explain better this part as it is better going for the garment mesh: However, this approach proved flawed because the GT garment mesh does not represent a direct visualization of the sewing patterns sewn and draped over a body model . Instead, it often deviates from the theoretical garment structure due to inaccuracies in the backbone model's predictions. **RM::** Questa parte la riscriverei mettendo che invece di "c'era un errore", più tipo "one might think that it is sufficient to... . However, this is not possible because Then, we decided to ... To rectify this, the approach shifted our focus from sewing patterns to the garment mesh—specifically, the version of the garment where the sewing patterns have been physically simulated and draped

over a human body model. This transition was made feasible by leveraging the recent advancements in Warp, a physics-based simulation engine, as demonstrated in the work of Maria et al.⁽¹⁾.

Draping - Warp

In this stage, the JSON file containing the sewing pattern specifications is fed into Warp, a high-performance physical simulation engine. Initially, the individual 2D panels are placed around the SMPL model according to the rotation and translation vectors defined in the file. For instance, the `top_front` panel is positioned over the chest area, while other panels like sleeves and back sections are similarly arranged based on the provided specifications. After positioning the panels, a bounding box mesh is created to define the sewing constraints, facilitating the correct stitching of panels.

Once the panels are positioned, Warp initiates the draping simulation by applying physical forces such as gravity and collision detection. This simulation allows the garment to naturally conform to the body's contours, accurately representing how the fabric would drape and settle in a real-world scenario. At the end of this process, Warp outputs only the garment mesh without the body model, providing a realistic 3D representation of the draped garment. Additionally, Warp provides two important pieces of information:

1. A **segmentation file** that tracks each vertex's panel of origin. Since the vertices were originally assigned to specific panels, this information can be preserved throughout the draping process.
2. An **UV map** that corresponds to the sewing patterns, which will later be used for refining the sewing patterns based on the draped garment mesh.

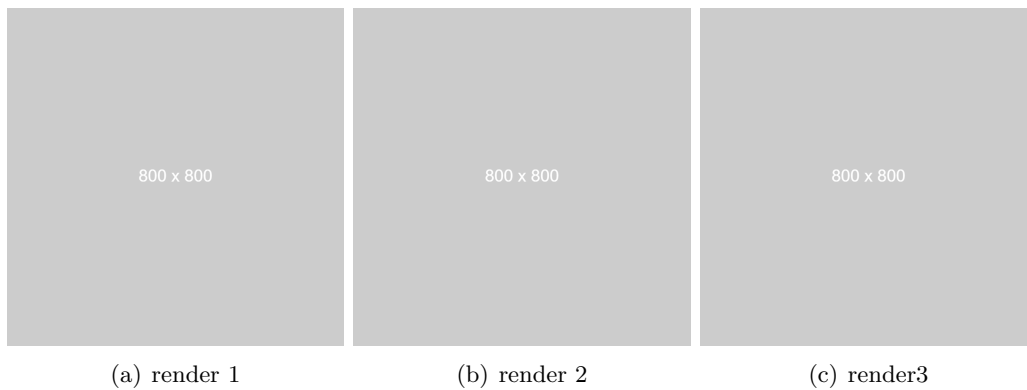


Figure 4.18 TBD: renders of warp after the physics simulation

A-Pose vs T-Pose

One challenge encountered during this process is the different poses used by standard SMPL models across datasets. Typically, datasets utilize the **T-Pose**, where the

subject stands with arms extended horizontally, facilitating a consistent framework for garment fitting and analysis. Conversely, Warp operates using the **A-Pose**, where the arms are positioned slightly downward, more closely resembling a natural standing posture.

This discrepancy between the T-Pose and A-Pose can lead to discrepancies in garment representation, even when the predictions are accurate. The backbone model’s rotation and translation matrices are calibrated based on the T-Pose assumption. However, when these parameters are applied to an A-Pose SMPL model within Warp, the resulting garment mesh may exhibit misalignments and unnatural folds. For example, in a T-Pose, the fabric around the neck and shoulders might stretch differently compared to an A-Pose, where the folds and tension in the garment change due to the altered arm position, leading to visual distortions and inaccuracies.

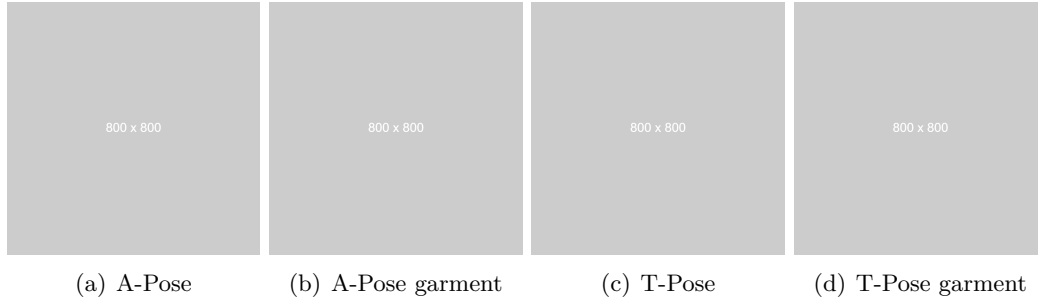


Figure 4.19 TBD: differences between A and T pose

Attempting to change the pose of Warp’s SMPL model from A-pose to T-pose is not trivial. In `GarmentCodeData`⁽⁹⁾ they focus on accurate garment measurement, meaning any modifications to the SMPL model would also require changes in the entire pipeline, from the rotation and translation matrices to the underlying code specific to the A-pose. Since garment refinement is more concerned with accuracy at the panel level, these minor discrepancies between T-Pose and A-Pose do not have a major impact on the final result. Thus, the refinement process can proceed without significant changes to Warp’s default A-Pose setup.

Garment Reconstruction

The final task is to shift the Gaussian Splatting focus from reconstructing the 2D sewing patterns to reconstructing the draped garment mesh. The Gaussian Splatting setup remains unchanged—cameras are still positioned around the garment, and images are captured from multiple viewpoints. The primary difference is that the reference images ($x^i \hat{x}_i x^i$) are no longer of the 2D sewing patterns but instead the 3D garment mesh output from Warp. Figure 4.20 shows some of the reference images captured by the cameras, which provide a comprehensive view of the garment mesh

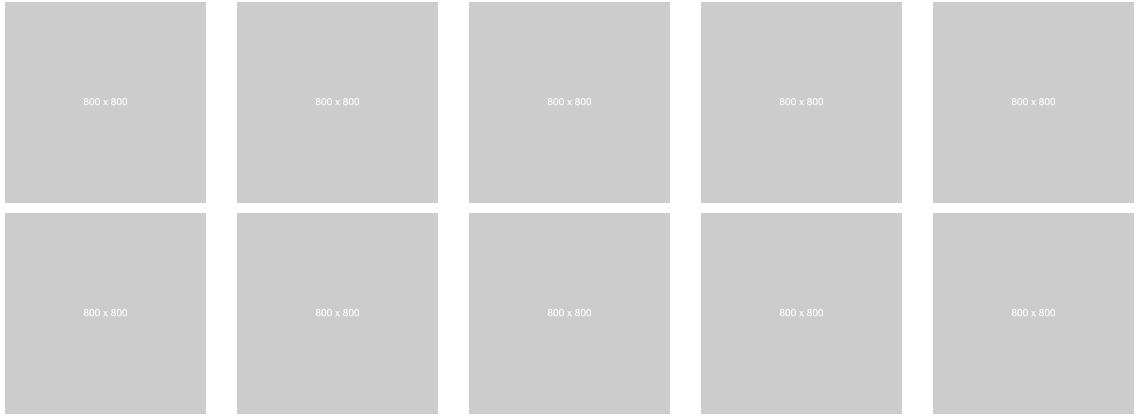


Figure 4.20 Input Reference Images

The Gaussian splatting algorithm then reconstructs the garment mesh by adjusting the Gaussians to align with the reference images. Just like in the previous steps, the Gaussians evolve over time, moving, resizing, and altering their opacity to minimize the difference between the generated renders and the reference images. The result is a reconstructed garment mesh that closely matches the ground truth, as shown in Figure 4.23.

Normals Inconsistency

One challenge encountered during this reconstruction phase involved the normals of the garment mesh. In some cases, the render failed to accurately capture the internal regions of the mesh, particularly omitting faces where the normals were aligned with the camera's viewing rays. This issue led to incomplete or incorrect renders (as depicted in Figure 4.21), where entire sections appeared missing or distorted in the rendered images.

To address this, the vertices were duplicated, and the corresponding faces were inverted to correct the normal directions. This simple solution ensured that both the internal and external parts of the garment were properly rendered (Figure 4.22).

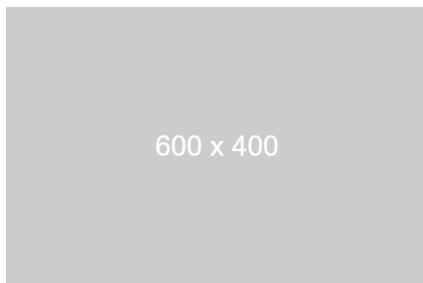


Figure 4.21 TBD: wrong render

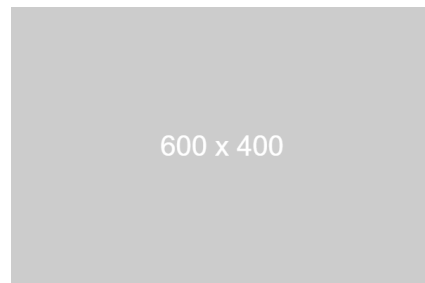


Figure 4.22 TBD: true render

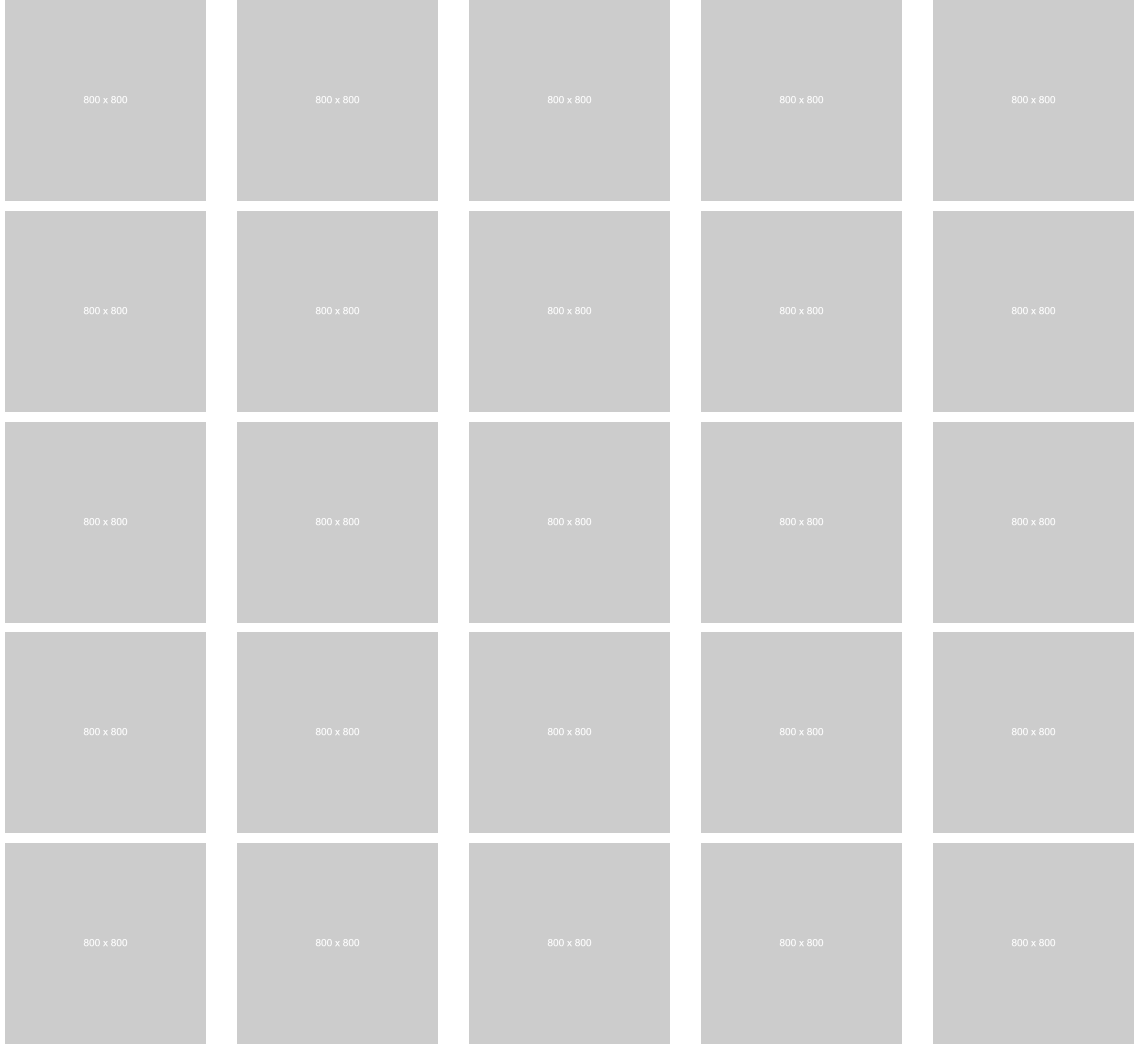


Figure 4.23 Output Images of Gaussian Splatting

4.2 Refinement in GS

In the second phase of the pipeline, the focus shifts from the initial reconstruction of the garment mesh to refining the representation of the garment. The primary objective of this refinement stage is to adjust the Gaussian splats—initially positioned based on the reconstructed garment mesh—to better align with the ground truth (GT) garment data. This alignment is essential for correcting inaccuracies that may arise from the backbone model’s predictions. The core idea is to switch the reference images used by the Gaussian splatting algorithm. During the first stage (reconstruction), the target was the garment mesh produced by Warp. Now, in the refinement stage, the target is updated to be the ground truth garment mesh, and the Gaussian splatting model uses the same setup but with different reference images (`imag_cond`) for each camera. This allows the algorithm to progressively adjust the Gaussians, refining the garment mesh to correct any errors present in the backbone

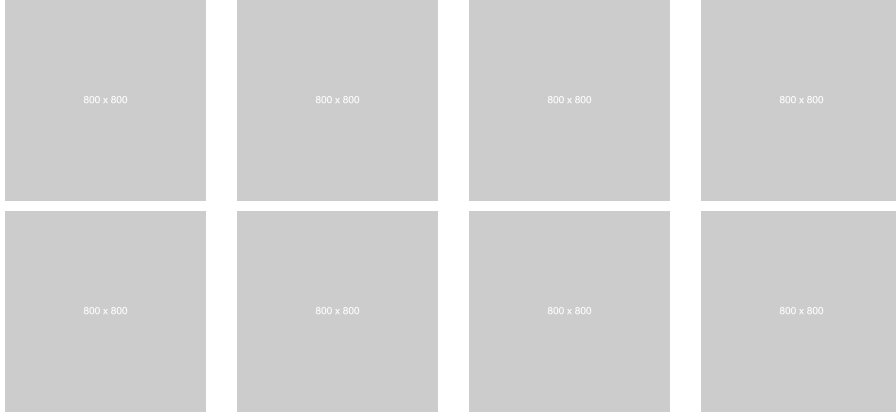


Figure 4.26 GS Reconstruction and Refining Process

model's initial prediction.

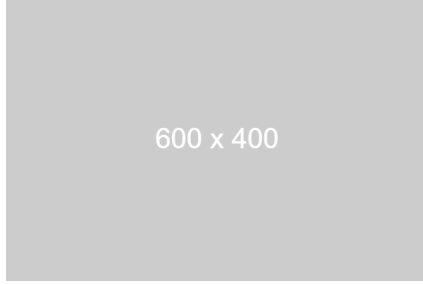


Figure 4.24 TBD: render of the warp garment

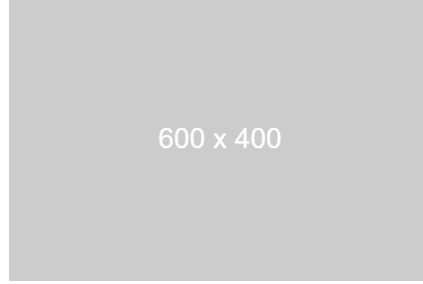


Figure 4.25 TBD: render of the GT garment

To implement this, a single GS model is used throughout the entire process. When first setting the Gaussian Splatting environment two sets of renders are created using the same camera configuration: one using the garment mesh $\hat{\mathcal{G}}$ generated from the Warp simulation, and another using the GT garment data $\mathcal{G}$. During the initial reconstruction phase, a set number of `cold_start` steps are dedicated to constructing the garment mesh. As the refinement begins, the training parameters are reset, and the reference images used by the cameras are switched to those of the GT garment mesh. This modification enables the GS Refiner to iteratively adjust the Gaussian splats—through movements, scaling, and rotations—to morph the reconstructed mesh towards the GT garment geometry over a defined number of `edit_steps` (as shown in figure 4.26).

4.2.1 Opacity Problem

One of the key issue in this refinement process involves the management of opacity in Gaussian Splatting. The idea seems straightforward: when refining certain areas of the garment (such as converting a U-neck into a V-neck), the Gaussians that are no longer necessary should simply have their opacity set to zero, effectively "removing" them from the scene. For example, if a garment's neckline should be

more pronounced, the Gaussian splatting algorithm could reduce the opacity of the Gaussians around the neck, creating the correct neckline and updating the UV map to remove points that correspond to invisible Gaussians. However, this approach introduces a significant challenge. The physical simulation that creates the draped garment (using Warp) introduces randomness, meaning that even two identical garments can have slightly different draping due to varying physical perturbations. This results in different Gaussian placements, even when they are theoretically the same.

As a result, when opacity is adjusted, not only are the unwanted areas removed, but also many Gaussians that have been displaced due to these draping variations. This leads to an unintended loss of Gaussians, resulting in a significantly reduced number of points in the final representation (Figure 4.27).

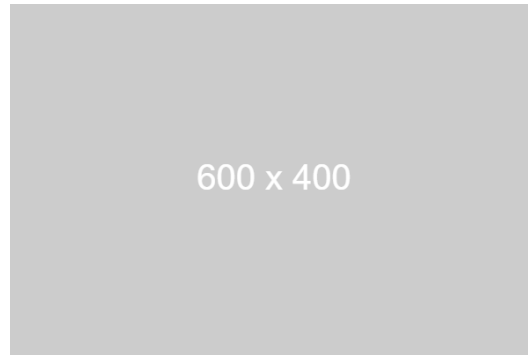


Figure 4.27 TBD: Opacity

To address this, the opacity adjustment feature is disabled during refinement. Instead, only movements, scaling, and rotations are permitted. This ensures that the Gaussians are not entirely removed but are allowed to adjust their positions, ensuring that the same number of points exists before and after the refinement process. Movement is prioritized by setting its learning rate higher than scaling and rotation, as the latter two are less critical for generating an accurate correspondence between the sewing patterns and the final garment.

Displacement Map

During the refinement process, it becomes crucial to track the movement of each gaussian in the mesh. One way to visualize this movement is by extracting the XYZ positions of the gaussians at each stage and creating a point cloud that represents both the initial and refined garment meshes. By visualizing these point clouds, it is possible to compare the two meshes and assess how effectively the refinement process is aligning the reconstructed garment with the GT mesh. Tracking the movement of the Gaussians during refinement provides critical insights into how the mesh evolves.

One useful visualization technique is to color the Gaussians based on a rainbow colormap before refinement and maintain the same colors after refinement (Figure

4.28(a and b). If the final mesh appears smooth and retains a consistent color distribution, this indicates that the refinement process has maintained coherence. However, if the color distribution is noisy or chaotic (this was the case where the initial point cloud was randomly initialized), it suggests that the movement of Gaussians is less structured, making it difficult to track their movement precisely.

A more detailed visualization can be created by assigning colors to each Gaussian based on their displacement during refinement. For example, a red-to-green colormap can indicate how far each Gaussian has moved, with red representing points that have moved the furthest and green representing points that moved the least. This involves calculating the displacement vector $\mathbf{d}$ for each Gaussian splat by comparing its position before and after refinement. Mathematically, the displacement $\mathbf{d}_i$ for each Gaussian splat θ_i can be defined as the vector difference between its current position $\mathbf{p}_i$ and its corresponding position in the GT garment mesh $\mathbf{p}_i^{\text{GT}}$:

$$\mathbf{d}_i = \mathbf{p}_i^{\text{GT}} - \mathbf{p}_i \quad (4.15)$$

This vector quantifies the movement of each splat during the refinement process, providing a basis for subsequent adjustments aimed at aligning the splats with the GT garment mesh. This technique highlights key areas of the garment where the refinement has made the most significant impact, such as a neckline that has been reshaped (Figure 4.28). The visualization clearly shows areas, such as the neckline, where significant adjustments were made, corresponding to regions where the backbone model's predictions deviated from the GT.

In some cases, an exponential displacement map (Figure 4.28 (c)) is used to exaggerate small changes, making it easier to visualize areas that might have undergone less obvious refinements:

$$\mathbf{d}_i^{\text{exp}} = e^{\|\mathbf{d}_i\|} \cdot \mathbf{d}_i \quad (4.16)$$

The application of the Displacement Map addresses a fundamental challenge in the refinement process: the variability introduced by the draping simulation. This is evident in the displacement map representation, particularly in the skirt, where, the draping introduces variations, creating displacement patterns that must be accounted for due to the influence of fabric folds.

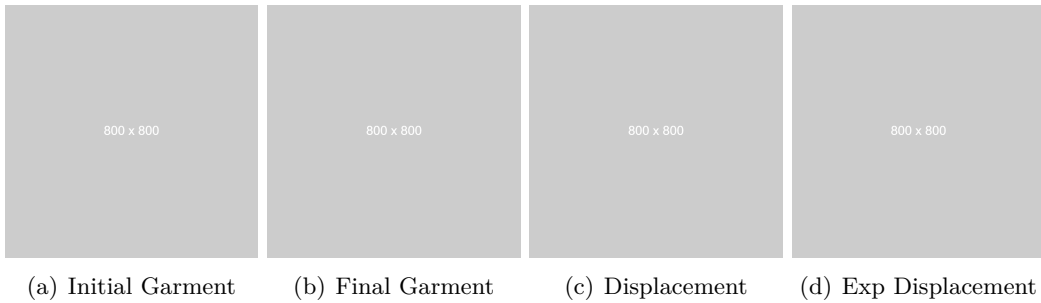


Figure 4.28 TBD: Visualization of the refining process

4.3 Optimizations

In this section, several optimizations applied to the basic pipeline are discussed. These optimizations aim to either improve the final accuracy of the reconstruction or reduce the computational cost, speeding up the overall process. Each optimization is validated in the 5.1 section, where the benefits are empirically demonstrated.

Mesh Loss

A primary challenge in the initial GS reconstruction phase was the inaccurate representation of garment panels. Specifically, Gaussian splats tended to form thick clusters rather than thin, precise layers that accurately represents the garment’s surface. This creates a misleading result, where the render might look correct from one perspective but is incorrect in three-dimensional space, resembling a block-like structure rather than a garment.

To address this issue, a Mesh Loss $\mathcal{L}_m$ is introduced. This loss encourages the Gaussians to align more closely with the actual surface of the garment mesh. The Mesh Loss is mathematically defined as the Mean Squared Error (MSE) between the positions of the Gaussian splats Θ_i and the corresponding points on the garment mesh $\hat{G}_i$:

$$\mathcal{L}_m = \frac{1}{N} \sum_{n=1}^N \|\Theta_i - \hat{G}_i\|^2 \quad (4.17)$$

where N represents the total number of Gaussian splats and garment mesh points. This loss function ensures that each Gaussian splat is as close as possible to its designated mesh point, promoting a thin and accurate representation of the garment panels.

To prevent premature convergence of the Gaussians to the surface—potentially disrupting the unity of the overall structure—this loss is applied only after a predefined number of iterations, allowing the Gaussian splats to first establish a coherent distribution before fine-tuning their alignment with the mesh. This optimization significantly improves the quality of the final point cloud representation, as shown in Figure ??.



Figure 4.29 TBD: PCD w/o Mesh Loss



Figure 4.30 TBD: PCD w Mesh Loss

Neighbor Loss

In addition to addressing the thickness of Gaussian splats, it was essential to maintain spatial coherence among neighboring splats to mimic the physical properties of fabric. In real-world garments, the fibers within the fabric tend to maintain consistent distances from one another unless acted upon by external forces (e.g., stretching or pulling). To replicate this behavior within the GS framework, a Neighbor Loss $\mathcal{L}_{knn}$ was introduced. The Neighbor Loss aims to preserve the relative distances between each Gaussian splat and its k -nearest neighbors (KNN). This loss encourages Gaussian splats to maintain consistent spacing, preventing unnatural clustering or dispersion that could distort the garment’s appearance.

Initially, during training, the distance between each Gaussian and its k -nearest neighbors is computed. This distance is then recalculated periodically (every `delta_n` steps), and the difference is backpropagated to the network to encourage consistency:

$$\mathcal{L}_{knn} = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^k \|\Theta_i - \Theta_{knn(j)}\|^2 \quad (4.18)$$

where $\Theta_{knn(j)}$ denotes the j -th nearest neighbor of the i -th Gaussian splat. After a certain number of training steps, the Neighbor Loss was gradually phased out, permitting the splats to adjust their positions more freely to accurately refine the garment’s geometry.

Cutting the First Stage

To optimize the reconstruction pipeline’s efficiency, it was observed that the initial "cold start" phase was redundant given the informed starting point provided by the mesh-sampled point cloud. Initially, the pipeline involved a two-step process where the Gaussians first reconstructed the initial garment mesh before refining it to match the ground truth mesh. However, since the purpose is to capture the displacement between the initial garment and the ground truth garment, the points sampled from the garment mesh $\bar{\mathcal{G}}$ already provide the necessary spatial information.

By skipping the reconstruction of the initial garment and directly starting with the displacement-based refinement, the pipeline can be accelerated significantly—by approximately 100%. As demonstrated in the validation analysis section 5.1, the final results are not compromised by cutting this stage, while the computational time is drastically reduced, making the entire pipeline more efficient.

Segmentation

One of the most impactful optimizations comes from the use of segmentation maps. In the initial implementation, the Gaussian Splatting process did not take advantage of the panel segmentation information available for each vertex in the garment mesh. However, both the garment mesh $\bar{\mathcal{G}}$ and the ground truth mesh $\hat{\mathcal{G}}$ include segmentation information that specifies which panel each vertex belongs to. This information allows the Gaussian Splatting process to be divided into smaller, more

manageable subproblems. The segmentation information is incorporated into the Gaussian Splatting by assigning a Panel ID attribute to each Gaussian splat.

During training, the cameras used for rendering also generate reference images for each individual panel (with black and white backgrounds) (Figure 4.31), and the Gaussian Splatting process switches reference renders based on the current panel being refined. The loss computation and backpropagation are then restricted to splats within a specific panel, preventing cross-panel interference and maintaining the distinct characteristics of each garment component.

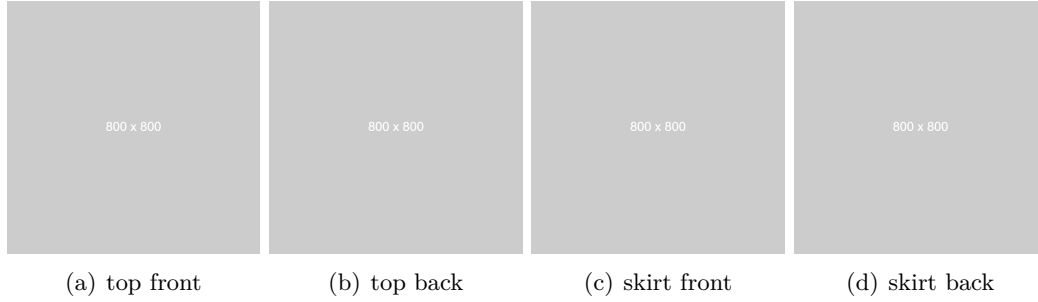


Figure 4.31 TBD: Segmented Renders

However, the use of segmentation maps also introduces new challenges. The accuracy of the segmentation information from both the Warp-generated garment mesh and the ground truth mesh can sometimes be imperfect, leading to mislabeled vertices. These misclassifications can create issues during the refinement process, as mislabeled vertices might not receive the correct adjustments, resulting in incomplete refinement of the garment (Figure ??). For example, if some vertices are incorrectly labeled, gaps or overlaps can occur in the final rendered garment.



Figure 4.32 TBD: Mesh w/o optimization



Figure 4.33 TBD: Mesh w optimization

To mitigate this, an algorithm was developed to enhance the accuracy of the segmentation maps. This algorithm examines the labels of each vertex's neighboring vertices and reassigns the label based on the majority class among the neighbors if the initial label does not meet a predefined threshold. Formally, for each vertex v_i :

$$S_i = \begin{cases} \text{Majority Class of Neighbors,} & \text{if majority} > \text{threshold} \\ S_i, & \text{otherwise} \end{cases} \quad (4.19)$$

The integration of segmentation maps offers several benefits:

1. **Focused Optimization:** By treating each garment panel as a separate subproblem, the refinement process can apply specialized adjustments that cater to the unique characteristics of each panel.
2. **Structural Integrity:** Segmentation ensures that the refinement of one panel does not inadvertently affect adjacent panels, preserving the garment's overall structure.
3. **Enhanced Fidelity:** Panel-wise optimization allows for finer control over the refinement process, leading to more accurate and realistic garment representations.

This optimization not only improves the visual quality of the reconstructed garment but also facilitates easier downstream processing, such as correspondence mapping between the 3D mesh and 2D sewing patterns.

4.4 Correspondance Mapping

Mapping displacements from 3D space back onto the 2D plane of sewing patterns presents one of the core challenges of this pipeline. Once the 3D displacement has been computed through the Gaussian Splatting refinement, the challenge lies in translating this into meaningful adjustments within the 2D sewing pattern domain through UV coordinates.

UV coordinates are particularly useful here as they provide a consistent 2D mapping of the garment's 3D surface, independent of draping or other physical transformations applied during simulation. This fixed correspondence, generated in the Warp-based draping phase, ensures that each vertex in 3D space $P_{3D} = \{p_i^{(x)}, p_i^{(y)}, p_i^{(z)}\}_{i=1}^N$ has a fixed corresponding UV coordinate $P_{2D} = \{p_i^{(u)}, p_i^{(v)}\}_{i=1}^N$.

Following refinement, each Gaussian Splatting point moves from its initial position P_i to a new target position $\hat{P}_i$. The displacement vector D_i of this movement, defined as:

$$D_i = \hat{P}_i - P_i \quad (4.20)$$

provides directional and magnitude data for each 3D point adjustment.

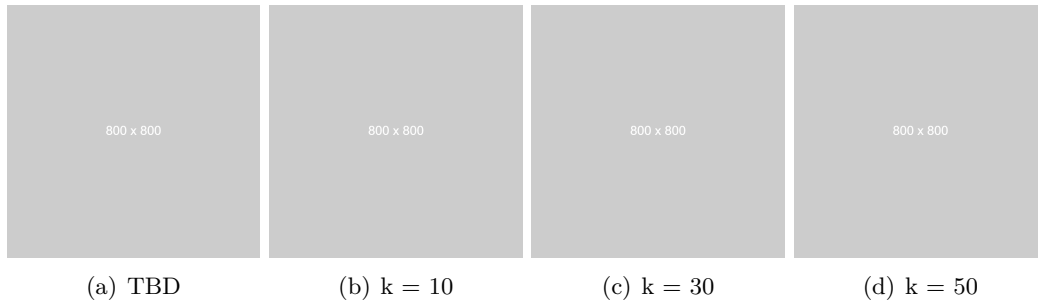


Figure 4.34 TBD: Displacements

The challenge now is to transfer these displacements onto the UV plane in a way that respects the panel’s structural integrity, maintaining both accurate displacement direction and magnitude. To achieve this, a suitable 2D displacement vector $D_i^{(2D)}$ is computed for each point, yielding a final refined sewing pattern representation $\hat{P}_{2D} = \{\hat{p}_i^{(2D)}\}_{i=1}^N$, where:

$$\hat{p}_i^{(2D)} = p_i^{(2D)} + D_i^{(2D)} \quad (4.21)$$

The sections below explore several techniques for translating 3D displacements into the 2D space, each with its own advantages and limitations.

4.4.1 Flatting Techniques

The process of converting the garment’s 3D structure into a flattened 2D form is complicated by the influence of fabric draping, which causes non-uniform deformation. In a physical garment, the same sewing pattern may produce varying drapes depending on factors like the wearer’s movement or environmental conditions. Variations in garment draping can result in shifting 3D displacements that are not always relevant to the final 2D pattern. For example, areas that shift due to fabric folds should ideally maintain stable distances, as in real fabric structure, regardless of draping changes. To accommodate this, the neighbor loss optimization discussed in section 4.3 provides a framework for emulating real-world textile behavior, preserving neighbor relationships and producing a smoother 2D map.

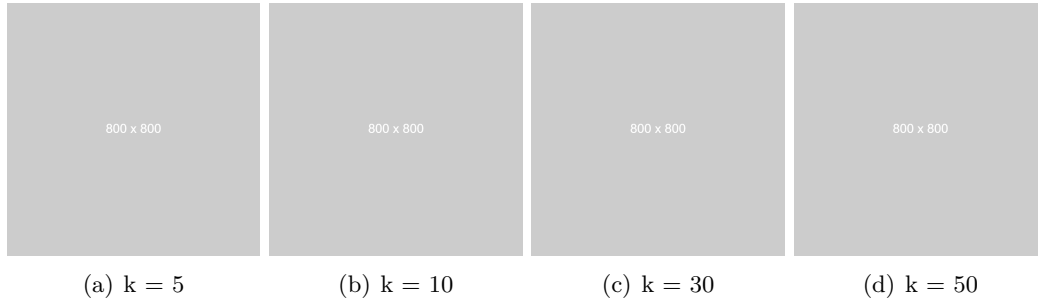


Figure 4.35 TBD: knn

Using neighbor distance constraints helps mitigate some inconsistencies in garment structure, but a reliable method of preserving a consistent 2D projection remains essential. Consequently, the goal of flattening is to project the 3D displacements onto the 2D sewing pattern while disregarding unnecessary draping effects.

Direct Projection

The most straightforward approach to flattening a 3D garment mesh is Direct Projection, wherein one of the spatial dimensions—typically the zzz-axis—is removed to project the 3D points onto the 2D plane. Mathematically, for each point $P_i = (x_i, y_i, z_i)$ in the 3D space, the corresponding 2D point $P_i^{(2D)}$ is obtained by:

$$P_i^{(2D)} = (x_i, y_i) \quad (4.22)$$

While Direct Projection is computationally efficient and easy to implement, it suffers from significant information loss. By eliminating the z -coordinate, depth-related displacements and spatial relationships are irretrievably lost, leading to distortions particularly in areas with pronounced folds or curvature are perpendicular to the z -axis. For instance, points that are vertically aligned in 3D may collapse onto the same position in 2D, resulting in overlapping or coinciding points that deviate from the garment's true structure.

To illustrate (Figure 4.36), consider a simple example where 2D points are initially sampled within a circular boundary and then rotated along the z -axis. The direct projection of this rotated shape compresses the points onto a single plane, ignoring the rotation's impact and producing an inaccurate representation.

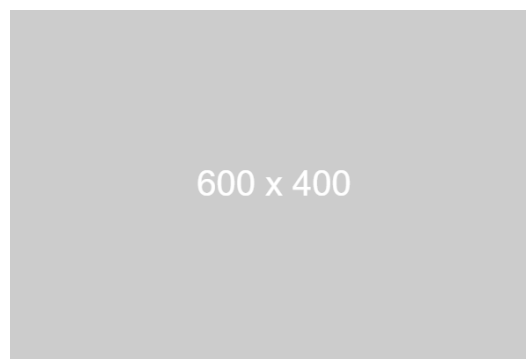


Figure 4.36 TBD: Opacity

Figure 4.37 illustrates the limitations of Direct Projection, where the refined sewing pattern appears distorted due to the loss of depth information. Despite its drawbacks, Direct Projection serves as a foundational method that can be augmented or combined with other techniques to mitigate its limitations.

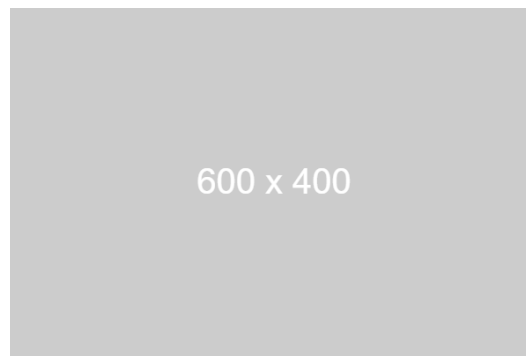


Figure 4.37 TBD: Opacity

Principal Component Analysis

Principal Component Analysis (PCA) provides a more sophisticated approach by finding the optimal projection plane based on the points' covariance matrix. PCA identifies the directions of maximum variance, or principal components, which

become the basis for reducing dimensionality. To achieve consistent 2D projections for the garment and refined meshes, the same two principal components must be used for both. This projection is ideal for relatively flat surfaces or panels with limited curvature, as PCA inherently selects the axes that best capture the garment's spatial variance.

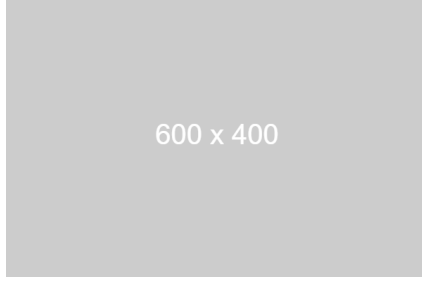


Figure 4.38 TBD: PCA 1



Figure 4.39 TBD: PCA 2

The figure above (??) shows PCA-flattened representations of the `top_front` and `skirt_front` panels for both the Warp-generated garment $\bar{\mathcal{G}}$ and the refined garment $\hat{\mathcal{G}}$. PCA effectively captures the primary orientation and spread of the garment, resulting in a more coherent and less distorted 2D sewing pattern compared to Direct Projection. However, PCA is inherently linear and may still struggle with garments that possess significant nonlinear deformations or complex fold patterns.

Jacobian Transformation

To achieve a more accurate correspondence mapping that accounts for the nonlinear deformations inherent in garment draping, the Jacobian Transformation technique is employed. The Jacobian Transformation is a more mathematically rigorous approach for converting 3D displacements into 2D coordinates by mapping surface displacements along tangent planes. By using the Jacobian, this method calculates how local 3D displacements translate to corresponding movements in UV space, effectively handling areas with complex deformations.

The Jacobian Transformation involves the following steps:

1. **Displacement Computation:** For each sampled point on the garment mesh, compute the 3D displacement vector $\Delta \mathbf{P}_i = \mathbf{P}_f - \mathbf{P}_i$, where $\mathbf{P}_i$ is the initial position and $\mathbf{P}_f$ is the final position after refinement.
2. **Triangle Identification:** Identify the triangular face within the mesh on which each point resides. This face is defined by vertices with known UV coordinates (u_0, v_0) , (u_1, v_1) , and (u_2, v_2) , corresponding to their 3D positions $\mathbf{P}_0$, $\mathbf{P}_1$, and $\mathbf{P}_2$.
3. **Jacobian Matrix Computation:** Compute the Jacobian matrix $\mathbf{J}$ that maps changes in the UV space to changes in the 3D space. For a point within a triangle, the 3D position $\mathbf{P}(u, v)$ can be expressed as:

$$\mathbf{P}(u, v) = \mathbf{P}_0 + \alpha(\mathbf{P}_1 - \mathbf{P}_0) + \beta(\mathbf{P}_2 - \mathbf{P}_0) \quad (4.23)$$

where α and β are the barycentric coordinates derived from (u, v) . The partial derivatives are:

$$\frac{\partial \mathbf{P}}{\partial u} = \frac{\partial \alpha}{\partial u}(\mathbf{P}_1 - \mathbf{P}_0) + \frac{\partial \beta}{\partial u}(\mathbf{P}_2 - \mathbf{P}_0) \quad (4.24)$$

$$\frac{\partial \mathbf{P}}{\partial v} = \frac{\partial \alpha}{\partial v}(\mathbf{P}_1 - \mathbf{P}_0) + \frac{\partial \beta}{\partial v}(\mathbf{P}_2 - \mathbf{P}_0) \quad (4.25)$$

The Jacobian matrix $\mathbf{J}$ is thus a 3×2 matrix composed of these partial derivatives.

4. **Projection onto Tangent Plane:** Compute the surface normal $\mathbf{N}$ at $\mathbf{P}_i$ using the cross product of the triangle edges. Project the 3D displacement $\Delta \mathbf{P}_i$ onto the tangent plane defined by $\mathbf{N}$:

$$\Delta \mathbf{P}_{\text{tangent}} = \Delta \mathbf{P}_i - (\Delta \mathbf{P}_i \cdot \mathbf{N})\mathbf{N} \quad (4.26)$$

5. **Adjust UV Coordinates:** Update the original UV coordinates $\mathbf{u}_i$ to obtain the refined UV coordinates $\hat{\mathbf{u}}_i$:

$$\hat{\mathbf{u}}_i = \mathbf{u}_i + \Delta \mathbf{u}_i \quad (4.27)$$

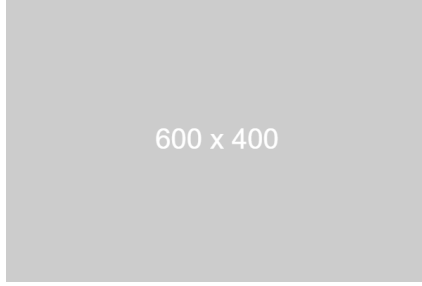


Figure 4.40 TBD: Jacobian



Figure 4.41 TBD: Expected

Despite the robustness of this approach, the practical improvement over PCA can be minimal, while computational complexity increases significantly. Consequently, PCA remains the preferred method in most scenarios due to its balance of efficiency and accuracy, as further detailed in the validation analysis in section 5.1.

4.5 No backbone

Francesco: Briefly explain that it is possible to remove the backbone and just using TailorGS Module, starting from a default panel. The only thing is that from a dress at least the right panels have to be chosen, i.e. if it has pants the default panel should be pants. This method has not been analyze in detail and panels have been set manually. The purpose is to show that is possible.

4.6 Implementation Details

Francesco: Like all the hyperparameters and the hardware used
hardware: CPU - intel 11900k GPU - 3090

Chapter 5

Results

5.1 Validation Analysis

Francesco: all the choices we made

5.1.1 Plots

5.1.2 Disappeared

5.1.3 Gaussians Displacement

Both the plots rainbow and displacement

5.2 Applications

Chapter 7

Conclusion

7.1 Key Contributions

7.2 GS in the garment reconstruction field

7.3 Limitations of TailorGS

7.4 Future Works

7.4.1 Flattening approach

Bibliography

- [1] Maria Korosteleva, Timur Levent Kesdogan, Fabian Kemper, Stephan Weninger, Jasmin Koller, Yuhan Zhang, Mario Botsch, and Olga Sorkine-Hornung. GarmentCodeData: A dataset of 3d made-to-measure garments with sewing patterns. In *Computer Vision – ECCV 2024*, 2024.
- [2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023.
- [3] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. *ICCV*, 2023.
- [4] Lijuan Liu, Xiangyu Xu, Zhijie Lin, Jiabin Liang, and Shuicheng Yan. Towards garment sewing pattern reconstruction from a single image. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 2023.
- [5] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Trans. Graph.*, 41(4):102:1–102:15, July 2022.
- [6] Miles Macklin. Warp: A high-performance python framework for gpu simulation and graphics. <https://github.com/nvidia/warp>, March 2022. NVIDIA GPU Technology Conference (GTC).
- [7] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-nerf 360: Unbounded anti-aliased neural radiance fields. *CVPR*, 2022.
- [8] Maria Korosteleva and Sung-Hee Lee. Neuraltailor: Reconstructing sewing pattern structures from 3d point clouds of garments. *ACM Trans. Graph.*, 41(4), 2022.
- [9] Maria Korosteleva and Sung-Hee Lee. Generating datasets of 3d garments with sewing patterns. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021.
- [10] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.

- [11] Georgios Pavlakos, Vasileios Choutas, Nima Ghorbani, Timo Bolkart, Ahmed A. A. Osman, Dimitrios Tzionas, and Michael J. Black. Expressive body capture: 3d hands, face, and body from a single image. In *Proceedings IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [12] Onur Ozyesil, Vladislav Voroninski, Ronen Basri, and Amit Singer. A survey of structure from motion, 2017.
- [13] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. SMPL: A skinned multi-person linear model. *ACM Trans. Graphics (Proc. SIGGRAPH Asia)*, 34(6):248:1–248:16, October 2015.
- [14] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *J. Vis. Comun. Image Represent.*, 18(2):109–118, April 2007.
- [15] L Paul Chew. Constrained delaunay triangulations. In *Proceedings of the third annual symposium on Computational geometry*, pages 215–222, 1987.
- [16] B Delaunay. 'delaunay triangulation. *Wikipedia*. Available at: http://en.wikipedia.org/wiki/Delaunay_triangulation (Accessed 9th July 2011), 1934.

Ringraziamenti

SCRIVI QUI I RINGRAZIAMENTI